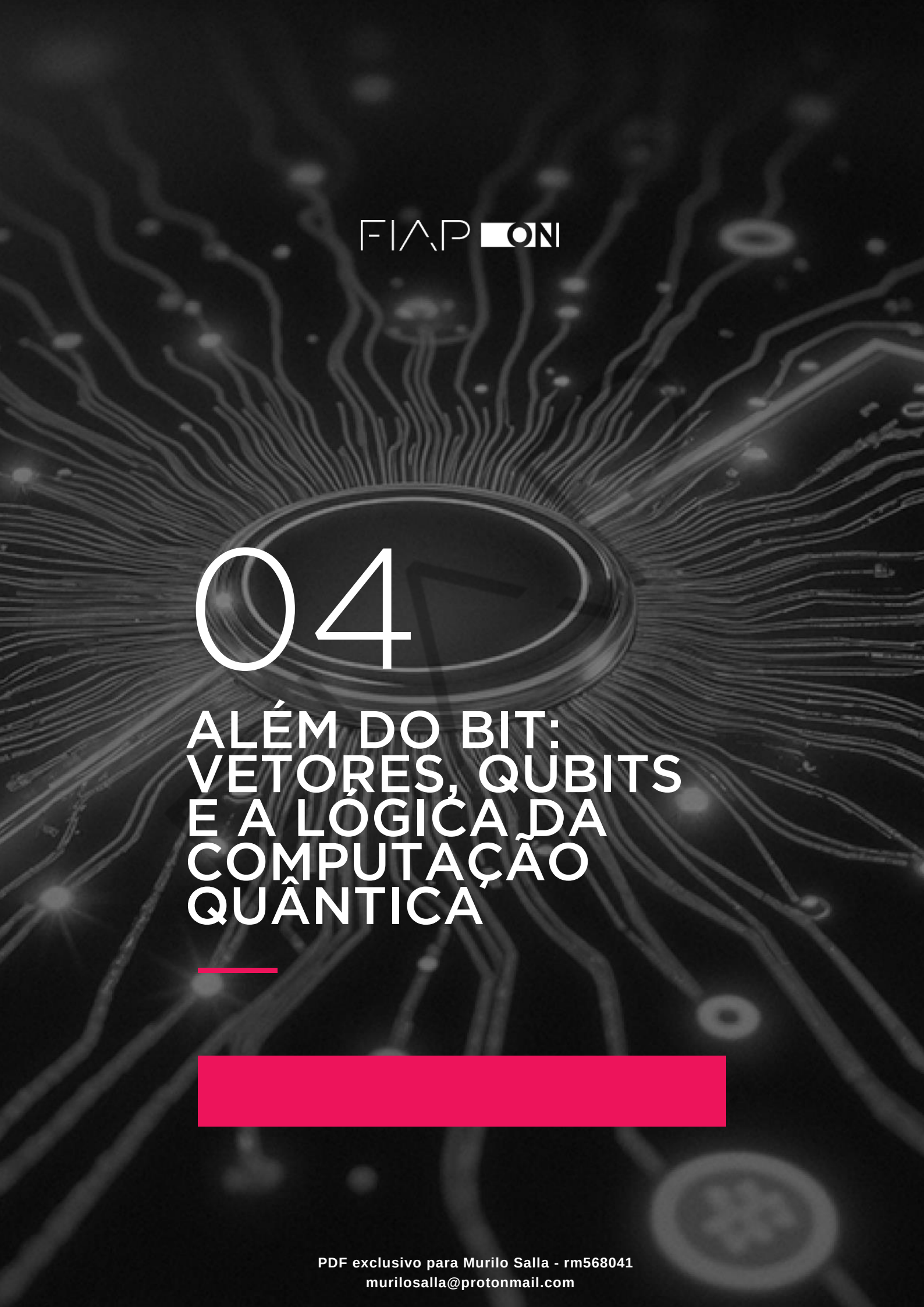


04

ALÉM DO BIT: VETORES, QUBITS E A LÓGICA DA COMPUTAÇÃO QUÂNTICA

PDF exclusivo para Murilo Salla - rm568041
murilosalla@protonmail.com

LISTA DE FIGURAS

Figura 1 - Completando quadrados (parte 1) 11
Figura 2 - Completando quadrados (parte 2) 11
Figura 3 - Completando cubos (etapa 1).....13
Figura 4 - Completando cubos (etapa 2).....14
Figura 5 - Construção geométrica de Descartes.....21
Figura 6 - Representação geométrica de F.....27
Figura 7 - Representação geométrica de v 28
Figura 8 - Representação geométrica de v e u 28
Figura 9 - Representação geométrica de v, u, w e a.....30
Figura 10 - Representação geométrica de v, vx e vy 32
Figura 11 - Representação geométrica dos múltiplos de v 35
Figura 12 - Estado zero na Esfera de Bloch.....53
Figura 13 - Estado um na Esfera de Bloch.....53
Figura 14 - Estado em superposição na Esfera de Bloch 54

FIAP

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Representando o vetor F em Python 26
Código-fonte 2 - Soma e subtração de vetores com NumPy 30
Código-fonte 3 - Decomposição de vetores em Python 32
Código-fonte 4 - Calculando a norma de um vetor em Python 33
Código-fonte 5 - Exemplos de multiplicação de vetor por escalar em Python 35
Código-fonte 6 - Exemplo de produto interno em Python 37
Código-fonte 7 - Exemplo de produto externo em Python 38
Código-fonte 8 - Exemplo de soma de matrizes e multiplicação por escalar em Python 40
Código-fonte 9 - Exemplo de multiplicação entre matrizes em Python 42
Código-fonte 10 - Exemplo de produto tensorial em Python 43
Código-fonte 11 - Exemplo de transposta de matriz em Python 43
Código-fonte 12 - Calculando autovetores e autovalores em Python 48
Código-fonte 13 - Criando vetores de estado em Qiskit 50
Código-fonte 14 - Aplicando transformações de estado em Qiskit 50
Código-fonte 15 - Calculando probabilidades em Qiskit 51
Código-fonte 16 - Visualizando a Esfera de Bloch em Qiskit 52

FINAL

SUMÁRIO

1 ALÉM DO BIT: VETORES, QUBITS E A LÓGICA DA COMPUTAÇÃO QUÂNTICA 6
1.1 Introdução 6

2 SOBRE OS NÚMEROS COMPLEXOS 7
2.1 A Matemática na Antiguidade 7
2.2 Notação de raiz quadrada 7
2.3 Herão de Alexandria e a altura da pirâmide 9
2.4 Cardano, Tartaglia, del Ferro e a Equação Cúbica 10
2.4.1 Scipione del Ferro e o método de completar cubos 10
2.4.2 O duelo entre del Fiore e Tartaglia 15
2.4.3 *Ars Magna* de Cardano 16
2.4.4 O problema da Fórmula de Cardano e a solução de Bombelli 17
2.5 Geometria da raiz de -1 20
2.5.1 Interpretação de René Descartes 20
2.5.2 O Rei da Matemática e os números complexos 21
2.5.3 O plano complexo de Wessel 22
2.5.4 Os avanços de Argand e Gauss 23
2.6 Conjugado de complexo 24

3 VETORES E ESTADOS DE UM QUBIT 25
3.1 O que é um vetor? 25
3.2 Propriedades básicas dos vetores 28
3.2.1 Soma e subtração de vetores 29
3.2.2 Decomposição de vetores 31
3.2.3 Módulo do vetor 32
3.2.4 Multiplicação de vetor por escalar 34
3.3 Multiplicação entre vetores 35
3.3.1 Produto Interno 36
3.3.2 Produto externo 37

4 MATRIZES E OPERADORES QUÂNTICOS 39
4.1 Propriedades básicas 39
4.2 Multiplicação de matrizes 41
4.2.1 Produto matricial 41
4.2.2 Produto tensorial 42
4.3 Transposta 43
4.4 Matriz Identidade 43
4.5 Matrizes complexas 44
4.5.1 Conjugado e Transposta Hermitiana 44
4.5.2 Matriz unitária 44
4.6 Notação de Dirac 45
4.6.1 Composição linear 46
4.6.2 Simplificando Produto Interno e Externo 46
4.7 Autovetores e autovalores 47

5 CRIANDO E MEDINDO ESTADOS DE QUBITS 49
5.1 Modificando um estado 50
5.2 Medindo um estado 51

PRELIMINAR

5.3 Visualização de estados.....52
6 CONCLUSÃO.....55
6.1 O que aprendemos com a Matemática 55
6.2 Da Matemática à Computação Quântica.....55
REFERÊNCIAS.....57

FIAP

1 ALÉM DO BIT: VETORES, QUBITS E A LÓGICA DA COMPUTAÇÃO QUÂNTICA

1.1 Introdução

A Computação Quântica utiliza como base os fenômenos da Mecânica Quântica, sendo necessário conhecer a matemática por trás destes conceitos para compreender algoritmos e circuitos quânticos.

Diferente dos bits clássicos, os qubits são representados por vetores em **espaços de Hilbert complexos de dimensão finita**, e a mudança em seus estados ao longo do tempo é descrita com **transformações lineares unitárias** nestes vetores.

Os números complexos são recorrentes nas equações da Mecânica Quântica. Eles são utilizados para descrever oscilações e fases de ondas, conceitos essenciais para representar os estados de um sistema e calcular as probabilidades associadas ao resultado de suas medições.

Já a álgebra linear nos fornece as ferramentas para manipular vetores (como os estados dos qubits), modelar transformações utilizando matrizes (o que é necessário para aplicar portas lógicas a circuitos quânticos) e calcular autovetores e autovalores (elementos centrais na medição do estado de um qubit).

Estes conceitos, que serão detalhados ao longo do capítulo, não são apenas ferramentas matemáticas: todos possuem interpretações e aplicações físicas diretas, utilizadas com frequência no trabalho com Computação Quântica.

2 SOBRE OS NÚMEROS COMPLEXOS

Para entender de fato a Computação Quântica, é necessário compreender o conjunto dos números complexos. Uma forma interessante para isso é estudar a história por trás destes números, desde os problemas que matemáticos da Antiguidade sofriam com o valor "impossível" $\sqrt{-1}$ até a aceitação e popularização do resultado "i".

2.1 A Matemática na Antiguidade

Durante muitos séculos, a Matemática foi vista como um conjunto de operações utilizadas para representar e resolver problemas do mundo real. Os matemáticos antigos resistiam à introdução de conceitos abstratos, e prontamente ignoravam resultados estranhos ou contraintuitivos. Entre eles, podemos destacar os números negativos e irracionais.

Antes da popularização da álgebra, durante a Idade Média, os problemas eram resolvidos de forma geométrica ou retórica, o que reforçava a ideia de que os resultados deveriam ser visualizáveis. Isso explica a desconfiança nos números negativos, já que não é natural pensar em valores menores que "nada", assim como nos números irracionais, que não podem ser corretamente representados pela Geometria.

Porém, às vezes apareciam resultados que até algebricamente são impossíveis de se interpretar, como a tão famosa $\sqrt{-1}$.

2.2 Notação de raiz quadrada

Antes de prosseguir, vale destacar o significado dos símbolos $\sqrt{\ }$, $+\sqrt{\ }$, $-\sqrt{\ }$, $\pm\sqrt{\ }$ e erros conceituais comuns.

Como todos sabem, a raiz quadrada é a operação utilizada para determinar um número que, quando multiplicado por si mesmo, resulta no número original. O que tendemos a esquecer é que sempre há dois valores que satisfazem essa relação: um

positivo e outro negativo (sendo “raiz de zero” a única exceção). Desta forma, a raiz quadrada de 4 pode ser tanto $+2$ quanto -2, já que:

$$(+2) \cdot (+2) = 4$$

$$(-2) \cdot (-2) = 4$$

Porém, o símbolo “$\sqrt{\ }$”, chamado de **raiz principal**, é equivalente a $+\sqrt{\ }$ e representa apenas o resultado positivo, ou seja, $\sqrt{4} = 2$, e apenas $+2$. Para considerar o resultado negativo ou ambos os resultados, é necessário indicar o sinal explicitamente. Assim:

$$\sqrt{4} = +\sqrt{4} = 2$$

$$-\sqrt{4} = -2$$

$$\pm\sqrt{4} = \pm 2$$

Logo, o modo correto de representar as equações é utilizando o sinal de “mais ou menos”, já que “$x^2 = y$” significa perguntar “qual número que, multiplicado por ele mesmo, resulta em y?”, portanto:

$$x^2 = y \leftrightarrow x = \pm\sqrt{y}$$

É importante notar que o sinal faz parte da raiz. Calcular a raiz negativa é o mesmo que calcular a raiz positiva e multiplicar o resultado por -1, destacando que:

$$-\sqrt{y} \neq \sqrt{-y}$$

Calcular a raiz de um número negativo consiste em admitir a existência de um valor que, multiplicado por ele mesmo, resulta em um valor abaixo de zero — algo contraintuitivo, já que até $(-1) \cdot (-1) = +1$, fazendo com que qualquer multiplicação entre números iguais seja sempre positiva.

Por isso, os matemáticos antigos tinham dificuldades em aceitar o resultado $\sqrt{-1}$, que é uma operação que não possui nenhum resultado **Real**, sendo impossível de ser interpretado.

2.3 Herão de Alexandria e a altura da pirâmide

Por meio da análise de documentos antigos, em especial os exemplos contidos no Papiro Matemático de Moscou (datado da era do Império Médio), é possível sugerir que os egípcios conheciam uma fórmula para calcular o volume do tronco de uma pirâmide quadrangular, semelhante à equação a seguir, em que a e b são as medidas dos lados das bases inferior e superior, respectivamente, e h é a altura da pirâmide.

Equação 1:

$$V = \frac{1}{3}h(a^2 + ab + b^2)$$

Apesar de correta, a fórmula utiliza diretamente a altura da pirâmide, o que é estranho, já que não é um comprimento que poderia ser facilmente medido na época. O mais intuitivo seria usar o apótema (altura da face lateral da pirâmide) no lugar, mas isso só foi feito no século primeiro pelo matemático Herão de Alexandria.

Herão, que é creditado como grego — embora existam hipóteses de que seja egípcio —, escreveu a altura da pirâmide em função da largura das bases e do apótema c, como mostra a equação a seguir.

Equação 2:

$$h = \sqrt{c^2 - 2\left(\frac{a-b}{2}\right)^2}$$

Embora funcione para a maioria dos casos, esta fórmula esconde um problema. Suponha que se deseje calcular a altura de um tronco de pirâmide cuja base maior mede $28\ m$, a base menor $4\ m$ e o apótema $15\ m$. Ao utilizar a **equação 2**:

$$h = \sqrt{15^2 - 2\left(\frac{28-4}{2}\right)^2}$$

$$h = \sqrt{225 - 2(12)^2}$$

$$h = \sqrt{225 - 288} = \sqrt{-63}\,m$$

Este não é o único exemplo de raízes de números negativos aparecendo como resultado da altura de uma pirâmide, o que gera uma contradição, já que as medidas de a, b e c são válidas e a pirâmide claramente possui uma altura.

Herão seria o primeiro matemático da história a utilizar raízes de números negativos para analisar um problema físico, porém, por considerar o resultado absurdo, contornou o problema usando o módulo do valor na raiz para calcular o resultado final. Ou seja, em vez de escrever:

$$h = \sqrt{225 - 288} = \sqrt{-63}$$

Herão considerava:

$$h = \sqrt{|225 - 288|} = \sqrt{+63} \approx 7.9m$$

O resultado está correto, mas a solução ignora a aparição da $\sqrt{-1}$ nos cálculos — como muitos matemáticos fizeram posteriormente ao se depararem com problemas semelhantes. Essa situação só começaria a mudar mais de mil anos depois.

2.4 Cardano, Tartaglia, del Ferro e a Equação Cúbica

Na época da Renascença Italiana, era comum cientistas proporem desafios intelectuais entre si. Matemáticos comumente elaboravam uma lista de exercícios para serem resolvidos em um duelo público; o vencedor poderia receber prêmios em dinheiro, uma vaga como professor universitário ou “apenas” propaganda de seus trabalhos e habilidades, o que atrairia a atenção de patrocinadores.

Como os antigos gregos já eram capazes de resolver equações de segundo grau por meios puramente geométricos, essas equações não representavam grandes desafios para eles. Equações cúbicas, por outro lado, eram praticamente impossíveis de serem resolvidas na época, já que não se conhecia nenhum algoritmo ou fórmula que sempre pudesse ser aplicado.

2.4.1 Scipione del Ferro e o método de completar cubos

Com intuito de ganhar vantagem nos duelos, o matemático Scipione del Ferro pensou em formas de adaptar o método dos gregos para equações de terceiro grau. Se conseguisse desenvolver tal algoritmo, Scipione se tornaria imbatível.

O método grego consistia em transformar o problema em uma soma de áreas, desenhando um quadrado que representasse a equação; ao encontrar o valor que completava corretamente o quadrado, a equação era considerada resolvida.

Um exemplo clássico para demonstrar o método é a resolução da equação:

$$x^2 + 26x = 27$$

Geometricamente, ela pode ser interpretada como sendo um quadrado de área x^2 (ou seja, lados iguais a x) e um retângulo de área $26x$ (podendo ser representado como tendo lados 26 e x), cuja união deve ser uma figura de área 27. Esta representação inicial do problema está ilustrada na figura “Completando quadrados (parte 1)”, em que o quadrado de lado x está em azul e o retângulo de área $26x$ está em verde.

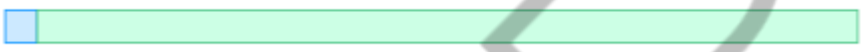


Figura 1 - Completando quadrados (parte 1)
Fonte: Elaborado pelo autor (2025)

Para resolver o problema, podemos manipular o retângulo, mantendo sua área, para formar um quadrado maior. Uma forma de se fazer isso é dividindo-o em duas partes, cada uma com área igual a $13x$.

Movendo um dos retângulos menores para que fique perpendicular ao quadrado de lado x, teremos uma figura similar a um “L”. Podemos “fechar” a nova figura ao desenhar um novo quadrado, que neste caso terá lados iguais a 13, ou seja, área $13^2 = 169$, representado em vermelho na figura “Completando quadrados (parte 2)”.

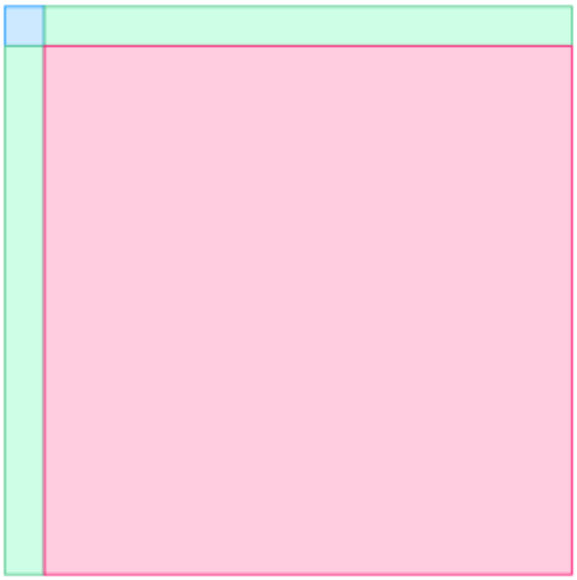


Figura 2 - Completando quadrados (parte 2)
Fonte: Elaborado pelo autor (2025)

Algebricamente, agora estamos resolvendo a equação:

$$x^2 + 2 \cdot 13x + 169 = 27 + 169$$

Note que a inclusão do termo 169 no lado direito da equação é necessária para que ela continue balanceada. Ao fatorar as expressões, temos:

$$(x + 13)^2 = 196$$

Ou seja, a nova figura representa um quadrado de área 196, cujos lados medem $x + 13$. Descobrir o x que resolve a nova equação é mais simples do que na equação original, permitindo sua resolução de forma puramente geométrica, apenas medindo os lados dos quadrados e retângulos.

Como $196 = 14^2$, é fácil ver que:

$$x + 13 = 14$$

$$x = 1$$

Conferindo na equação original:

$$1^2 + 26 \cdot 1 = 27$$

Resolver o problema puramente de forma algébrica nos mostra que, na verdade:

$$x = \frac{-26 \pm \sqrt{26^2 + 4 \cdot 27}}{2} = -13 \pm 14$$

$$x_1 = 1;\ x_2 = -27$$

Porém, como os resultados negativos eram considerados um absurdo, a solução x_2 era ignorada.

A ideia de del Ferro foi fazer o mesmo processo, mas para calcular o volume de cubos. Seu algoritmo, consideravelmente mais complicado do que o método grego, apesar de similar, era capaz de resolver equações cúbicas reduzidas, que não possuíam termo quadrático, de forma geral:

$$x^3 + px = q$$

Para ilustrar o método, considere o seguinte exemplo:

$$x^3 + 9x = 26$$

Neste caso, temos um cubo de volume x^3. Para iniciar o algoritmo, aumentamos os lados do cubo em y, formando um novo cubo de lados $x + y$ e volume $(x + y)^3$.

Para isso, desenhamos primeiro três paralelepípedos, com bases de área x^2 e altura y, resultando em um volume x^2y. Entre cada um dos pares de paralelepípedos, podemos desenhar um novo, de base y^2 e altura x, e que resulta em um volume xy^2.

Por fim, o espaço que resta entre os novos paralelepípedos pode ser preenchido por um cubo de lado y, tendo volume y^3. A figura final é o cubo de volume $(x + y)^3$. Esses passos iniciais estão ilustrados na figura “Completando cubos (etapa 1)”.

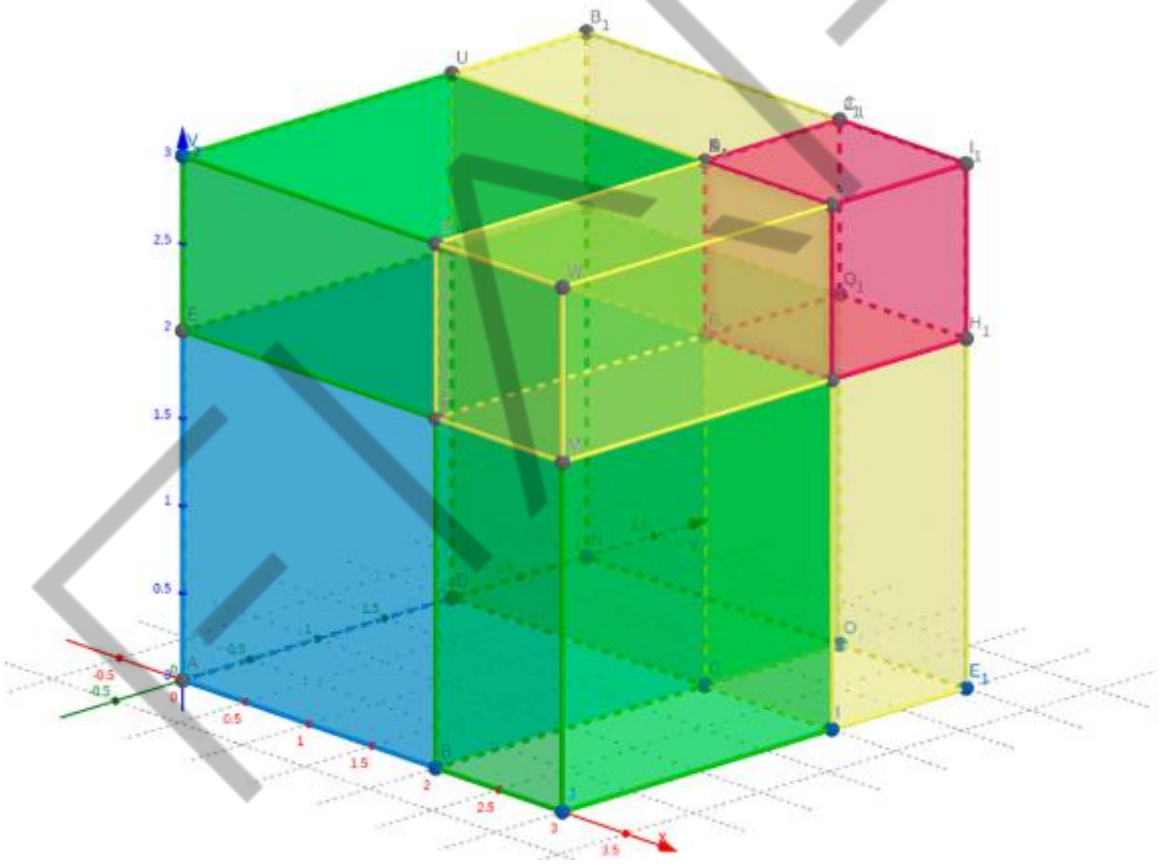


Figura 3 - Completando cubos (etapa 1)
Fonte: Elaborado pelo autor (2025)

Podemos dispor os seis paralelepípedos gerados para formar um novo paralelepípedo, de lados $x + y$, $3y$ e altura x, cujo volume é $3xy(x + y)$, como mostra a figura “Completando cubos (etapa 2)”.

Por simplicidade, seja $x + y = z$. Então, o volume do cubo formado na figura “Completando cubos (etapa 1)” é z^3, e o volume do paralelepípedo da figura “Completando cubos (etapa 2)” é $3xyz$.

Analisando a equação original, podemos associar o paralelepípedo de volume $3xyz$ com o termo $9x$, da mesma forma que fizemos com os retângulos de área $13x$ no exemplo anterior.

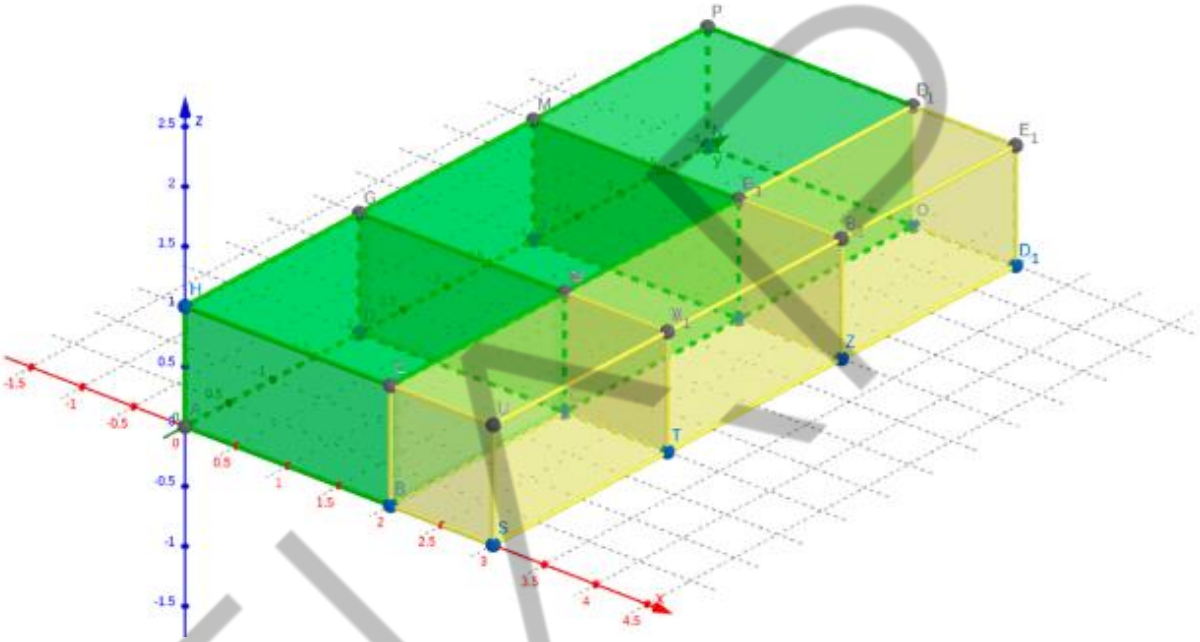


Figura 4 - Completando cubos (etapa 2)
Fonte: Elaborado pelo autor (2025)

Portanto, temos que:

$$3xyz = 9x \leftrightarrow yz = 3$$

Além disso, o volume do cubo formado na figura “Completando cubos (etapa 1)” é:

$$z^3 = x^3 + 9x + y^3$$

Substituindo valores, podemos montar o seguinte sistema de equações:

$$yz = 3$$

$$z^3 = 26 + y^3$$

Manipulando o sistema, eventualmente, é possível chegar no seguinte resultado:

$$y^6 + 26y^3 = 27$$

O que parece ter dificultado o problema, já que o grau da equação dobrou. Porém, seja $w = y^3$:

$$w^2 + 26w = 27$$

É fácil ver que, na verdade, o resultado é análogo ao exemplo utilizado para resolver equações quadráticas. Portanto:

$$w = 1 = y^3$$

$$z^3 = 26 + 1 \rightarrow z = 3$$

$$x + y = z \rightarrow x = 2$$

Assim, determinamos que a solução real para o problema original é 2, a única raiz que o método de del Ferro é capaz de encontrar.

2.4.2 O duelo entre del Fiore e Tartaglia

Como os conhecimentos mais avançados eram uma vantagem nos duelos, dificilmente os cientistas da época publicavam suas descobertas. Este foi o caso do algoritmo desenvolvido por del Ferro, que o registrou em forma de poema e enviou apenas para alguns amigos próximos. Um deles era seu discípulo, Antonio Maria del Fiore.

Del Fiore não era considerado um matemático brilhante para sua época, apesar de não ser ruim. Ele sabia resolver equações cúbicas reduzidas através do conhecimento passado por del Ferro enquanto era vivo, mas não era capaz de compreender o raciocínio que originou o algoritmo, sendo incapaz de melhorá-lo. Mesmo assim, utilizou o algoritmo de del Ferro para vencer duelos, chamando a atenção de outros matemáticos.

Por volta da mesma época, Niccolò Fontana — conhecido como “Tartaglia” (“Gago”, por sua dificuldade na fala causada por uma ferida de espada) — afirmava ter desenvolvido um método para resolver equações cúbicas da forma geral $x^3 + px^2 = q$, um avanço em relação a del Ferro. Por achar que Tartaglia estava blefando, del Fiore o desafiou para um duelo.

Prevendo que todos os problemas envolveriam equações cúbicas, Tartaglia passou as vésperas do duelo tentando descobrir o método de del Fiore.

Eventualmente, conseguiu redescobrir por conta própria o algoritmo de del Ferro, tornando-o capaz de resolver equações do tipo $x^3 + px = q$.

Tartaglia não estava blefando e realmente sabia resolver equações do tipo $x^3 + px^2 = q$. Aliado ao método de completar cubos que recém havia descoberto, foi capaz de resolver os 20 problemas propostos facilmente, enquanto del Fiore não foi capaz de resolver os mais difíceis propostos por Tartaglia, determinando o vencedor.

Como esperado, este duelo chamou a atenção de matemáticos importantes, como Cardano.

2.4.3 *Ars Magna* de Cardano

Girolamo Cardano foi um matemático brilhante e extremamente curioso. Ao saber do duelo, entrou em contato com Tartaglia para que este lhe ensinasse a resolução de equações cúbicas.

Pelos mesmos motivos de del Ferro, Tartaglia não queria divulgar sua equação. Além disso, ainda pretendia publicar um livro ensinando o que havia descoberto, porém, isso nunca aconteceu.

Após muita insistência, Tartaglia mostrou seus resultados a Cardano, com a condição de que fizesse um voto de silêncio. Embora tenha aceitado o acordo, Cardano buscava a demonstração completa da fórmula, não apenas a fórmula final.

Ao ouvir rumores de que Tartaglia não havia sido o primeiro a adquirir tal conhecimento, Cardano procurou del Fiore, que lhe mostrou os escritos deixados por del Ferro anos antes.

Com a demonstração do algoritmo de del Ferro, Cardano foi capaz de descobrir por conta própria a demonstração da fórmula que lhe havia sido mostrada por Tartaglia.

Como chegou aos resultados “sozinho”, Cardano se achou no direito de publicar ambas as fórmulas, com demonstrações completas, em seu livro chamado *Ars Magna*, de 1545. Mesmo dando créditos tanto para del Ferro quanto para Tartaglia, até hoje a fórmula para resolver equações de terceiro grau é comumente chamada apenas de “Fórmula de Cardano”.

Apesar das acusações de Tartaglia, que não aceitava a quebra do voto de silêncio, Cardano não era um plagiador. Além de creditar os descobridores originais das fórmulas para equações reduzidas, Cardano conseguiu melhorar a solução e encontrou uma fórmula capaz de resolver **qualquer** equação cúbica, de forma geral:

$$x^3 + a_1x^2 + a_2x + a_3 = 0$$

Para isso, Cardano provou que é sempre possível escrever uma equação cúbica na forma reduzida, utilizando uma troca de variáveis. A fórmula de Cardano-Tartaglia-Ferro, que resolve equações reduzidas do tipo $x^3 = px + q$, está ilustrada a seguir.

Equação 3:

$$x = \sqrt[3]{\frac{q}{2} + \sqrt{\frac{q^2}{4} - \frac{p^3}{27}}} - \sqrt[3]{-\frac{q}{2} + \sqrt{\frac{q^2}{4} - \frac{p^3}{27}}}$$

Porém, Cardano havia identificado um problema nesta equação, que a impossibilitava de resolver alguns exercícios.

2.4.4 O problema da Fórmula de Cardano e a solução de Bombelli

Suponha a seguinte equação cúbica reduzida:

$$x^3 = 15x + 4$$

Ao tentar resolvê-la utilizando a **equação 3**, temos:

$$x = \sqrt[3]{\frac{4}{2} + \sqrt{\frac{4^2}{4} - \frac{15^3}{27}}} - \sqrt[3]{-\frac{4}{2} + \sqrt{\frac{4^2}{4} - \frac{15^3}{27}}}$$

$$x = \sqrt[3]{2 + \sqrt{4 - 125}} - \sqrt[3]{-2 + \sqrt{4 - 125}}$$

$$x = \sqrt[3]{2 + \sqrt{-121}} - \sqrt[3]{-2 + \sqrt{-121}}$$

Novamente, os matemáticos se depararam com o problema da raiz de números negativos.

Em vez de afirmar que o problema em si estava errado, e por isso não tem solução, Cardano tentou resolver a equação por inspeção. Seja $x = 4$:

$$4^3 = 64$$

$$15 \cdot 4 \ + \ 5 = 60 + 4 = 64$$

Assim, mostramos que 4 é a raiz da equação. Cardano supôs que o problema estava na fórmula em si, dizendo que ela estaria incompleta ou apenas que não poderia ser aplicada a certos problemas.

Porém, outros matemáticos ficaram intrigados com o fato de que ao tentar resolver a equação, ela os levava para um resultado absurdo como $\sqrt{-1}$, sendo que claramente o problema tinha uma resposta real.

Alguns anos depois, em 1572, o engenheiro e arquiteto Rafael Bombelli publicou seu livro *L'Algebra*, no qual apresentou a mesma equação e sua tentativa de resolver utilizando a Fórmula de Cardano.

Com a popularização da Álgebra Moderna na Europa, permitindo que a Matemática se tornasse cada vez mais abstrata e distante da Geometria, Bombelli decidiu assumir $\sqrt{-1}$ como um número válido e continuou as contas, por mais que o consenso da época fosse que isso não fazia sentido prático.

Após algumas manipulações, Bombelli mostrou que:

$$x = \sqrt[3]{2 + \sqrt{-121}} - \sqrt[3]{-2 + \sqrt{-121}} = \sqrt[3]{2 + \sqrt{-121}} + \sqrt[3]{2 - \sqrt{-121}}$$

$$\sqrt[3]{2 + \sqrt{-121}} = 2 + \sqrt{-1}$$

$$\sqrt[3]{2 - \sqrt{-121}} = 2 - \sqrt{-1}$$

Portanto,

$$x = 2 + \sqrt{-1} + 2 - \sqrt{-1} = \mathbf{4}$$

Desta forma, Bombelli mostrou que o resultado real estava correto, utilizando a Fórmula de Cardano, que o próprio considerava ser inválida.

A partir deste momento, outros matemáticos passaram a estudar as propriedades do número $\sqrt{-1}$, buscando entender como um valor aparentemente

absurdo podia ser inserido em uma fórmula e, ainda assim, resultar em um valor existente.

No mesmo livro, Bombelli estrutura uma regra de sinais para as multiplicações entre ± 1 e $\pm\sqrt{-1}$:

> (...) fin che hò trovato la sua dimostrazione in linee (...) e prima trattarò del Moltiplicare, ponendo la regola del più & meno. [até que encontrei a sua demonstração em linhas (...) e primeiro tratarei do Multiplicar, pondo a regra do mais e menos.
>
> Più via più di meno, fa più di meno.
>
> Meno via più di meno, fa meno di meno.
>
> Più via meno di meno, fa meno di meno.
>
> Meno via meno di meno, fa più di meno.
>
> Più di meno via più di meno, fa meno.
>
> Più di meno via men di meno, fa più.
>
> Meno di meno via più di meno, fa più.
>
> Meno di meno via men di meno, fa meno.
>
> (BOMBELLI, 1572, p. 169)

Deixando de lado a tradução literal, que não faz total sentido, o trecho pode ser interpretado algebricamente como:

$$(+1)\cdot\left(+\sqrt{-1}\right) = +\sqrt{-1}$$

$$(-1)\cdot\left(+\sqrt{-1}\right) = -\sqrt{-1}$$

$$(+1)\cdot\left(-\sqrt{-1}\right) = -\sqrt{-1}$$

$$(-1)\cdot\left(-\sqrt{-1}\right) = +\sqrt{-1}$$

$$\left(+\sqrt{-1}\right)\cdot\left(+\sqrt{-1}\right) = -1$$

$$\left(+\sqrt{-1}\right)\cdot\left(-\sqrt{-1}\right) = +1$$

$$\left(-\sqrt{-1}\right)\cdot\left(+\sqrt{-1}\right) = +1$$

$$\left(-\sqrt{-1}\right) \cdot \left(-\sqrt{-1}\right) = -1$$

2.5 Geometria da raiz de -1

2.5.1 Interpretação de René Descartes

Apesar dos avanços de Bombelli, os cientistas ainda buscavam algum significado físico para $\sqrt{-1}$.

Em sua obra *La Géométrie*, publicada em 1637, René Descartes revolucionou a matemática ao introduzir diversos avanços, entre eles, a Geometria Analítica — que é a união entre álgebra e geometria. Essa abordagem permitiu tanto a resolução de problemas geométricos por meio da álgebra quanto a visualização e interpretação geométrica das equações algébricas.

Com essa união de conceitos, aliada ao sistema de coordenadas — também introduzido na mesma obra —, Descartes passou a buscar significado para alguns dos resultados estranhos com os quais matemáticos anteriores já haviam se deparado.

Entre eles, destaca-se a aceitação dos números negativos como resultado, interpretados como uma distância na direção contrária ao eixo de referência — um conceito considerado básico nos dias de hoje.

Outro resultado interessante foi a tentativa de resolver uma equação de segundo grau por construção geométrica. Diferente dos gregos, Descartes considerou que os coeficientes da equação eram distâncias entre pontos, como mostra a figura "Construção geométrica de Descartes", que ilustra a construção da equação $z^2 = az + b^2$, em que a e b^2 são valores não negativos.

Pela figura, é fácil ver que $OM = \frac{1}{2}a + \sqrt{\frac{a^2}{4} + b^2}$. Descartes provou que esta é a raiz positiva da equação, ou seja, $OM = z$. Como de costume para a época, a raiz negativa foi ignorada.

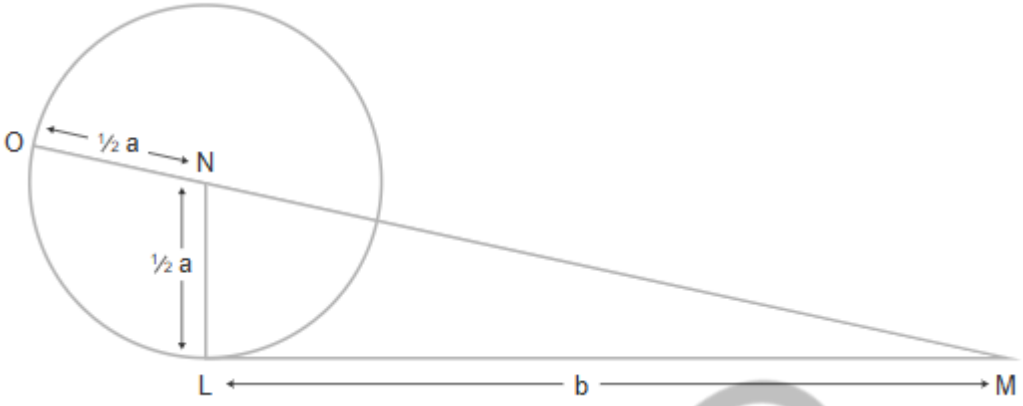


Figura 5 - Construção geométrica de Descartes
Fonte: An Imaginary Tale (1998), adaptado por FIAP (2025)

Depois, Descartes fez uma construção parecida para a equação $z^2 = az - b^2$, com as mesmas restrições, chegando na raiz:

$$z = \frac{1}{2}a \pm \sqrt{\frac{a^2}{4} - b^2}$$

Esta solução mostra as duas raízes da equação, positiva e negativa. Descartes as considerou válidas porque ambas podem ser desenhadas geometricamente.

Um detalhe importante de se notar é que, caso $b^2 > \frac{a^2}{4}$, ou seja, $b > \frac{a}{2}$, aparecerá uma $\sqrt{-1}$ na fórmula. Porém, devido a forma com que fazia suas construções, seria impossível desenhar a figura caso a relação fosse verdadeira.

Portanto, Descartes concluiu que $\sqrt{-1}$ é apenas um artifício algébrico: deve ser considerado para realizar cálculos, mas, se aparecer como resultado final, indica que a equação não tem significado físico.

2.5.2 O Rei da Matemática e os números complexos

Um século depois dos avanços de Descartes, Leonhard Euler, popularmente conhecido como Rei da Matemática, publicou seu artigo *De numeris imaginarium* (Sobre os números imaginários) em 1749.

Na obra, Euler discute os usos do número $\sqrt{-1}$ enquanto ferramenta algébrica, estendendo a notação de logaritmos que estava sendo desenvolvida para incluir os números negativos e múltiplos de $\sqrt{-1}$.

Além desta e de outras contribuições, Euler popularizou o nome e a notação que utilizamos até hoje. Concordando com Descartes — que afirmava que $\sqrt{-1}$ servia para indicar um resultado que não existe —, Euler batizou este valor de "número imaginário" ou "raiz imaginária", assim:

$$i = \sqrt{-1} \leftrightarrow i^2 = -1$$

Os múltiplos de i representam o conjunto dos números Imaginários Im, e o conjunto que representa a união dos Reais com Imaginários passou a ser conhecido como conjunto dos números Complexos $\mathbb{C}$, tal que:

$$C = \{a + bi \mid a \in R \; e \; b \in R\}$$

Assim, os números complexos possuem uma parte Real, com coeficiente $a \in R$, e outra parte imaginária, com coeficiente $b \in R$ multiplicado pela raiz imaginária.

Importante: para algumas áreas de estudos, a letra i já é amplamente utilizada para representar outros conceitos, como a corrente elétrica (Engenharia Elétrica/Eletrônica) ou um índice de iteração (Ciência da Computação). Por isso, nessas áreas, é comum utilizar a letra j para representar $\sqrt{-1}$ e, assim, evitar ambiguidades.

2.5.3 O plano complexo de Wessel

Apesar da crescente aceitação algébrica dos números complexos, matemáticos continuavam buscando uma forma de visualizá-los ou, ainda, dar uma interpretação física a eles.

Em 1797, o topógrafo Caspar Wessel publicou um artigo apresentando pela primeira vez uma representação geométrica clara para os números complexos.

No Plano Cartesiano, coordenadas são representadas por meio de dois eixos, ambos Reais. Wessel propôs o Plano Complexo — similar ao Plano Cartesiano, mas com um dos eixos Real e outro Imaginário. Assim, cada ponto no plano representaria um número complexo em sua forma $a + bi$.

Como será demonstrado nas próximas seções, Wessel provou, ainda, que multiplicar um vetor pela raiz imaginária causa uma rotação de noventa graus no sentido anti-horário, além de conceitos como a multiplicação de segmentos de retas e relações envolvendo a **forma polar** dos vetores.

Os avanços de Wessel consolidaram os números complexos na matemática. Porém, por ter publicado o seu artigo apenas em dinamarquês em uma revista com poucos leitores fora da Dinamarca, o trabalho de Wessel quase não teve nenhuma relevância na sua época. O artigo foi esquecido até 1895, quando foi encontrado na antiga biblioteca da academia.

2.5.4 Os avanços de Argand e Gauss

Em 1806, dois matemáticos redescobriram de forma autônoma os resultados de Wessel: o francês Abbé Adrien-Quentin Buée e o sueco Jean-Robert Argand.

Além de popularizar as ideias perdidas, que considerava os números complexos na forma **cartesiana** e **polar**, Argand ainda descobriu como representá-los na forma **exponencial**, expandindo os resultados de Wessel.

Décadas depois, em 1831, Carl Friedrich Gauss, conhecido como o Príncipe da Matemática, também publicou seus estudos envolvendo a interpretação geométrica dos números imaginários.

Foi Gauss quem popularizou o termo “números complexos”, além de ter sido quem os definiu formalmente, assim como suas propriedades algébricas e geométricas. Gauss também provou teoremas como o Teorema Fundamental da Álgebra, no qual afirma que qualquer equação polinomial com coeficientes complexos deve ter, no mínimo, uma solução complexa.

Diferente de Descartes e Euler, Gauss não concordava que $\sqrt{-1}$ fosse um valor “falso” ou que devesse ser ignorado. Para tentar aumentar ainda mais a aceitação deste conceito, sugeriu renomear o conjunto para “números laterais”, porém, sem muito sucesso.

O rigor matemático que Gauss trouxe foi tão importante para o estudo do número i, que o plano complexo idealizado por Wessel ficou popularmente conhecido até os dias de hoje como “Plano de Argand-Gauss”.

2.6 Conjugado de complexo

Como a Computação Quântica utiliza extensamente os números complexos, vale destacar uma de suas propriedades mais importantes: o “conjugado”. Como já mencionado, a forma geral de um número complexo é:

$$z = a + bi$$

O conjugado de z, denotado como z^C ou $\bar{z}$, é uma operação que representa uma reflexão do vetor z no plano de Argand-Gauss em torno das abscissas, ao trocar o sinal de sua componente imaginária:

$$\bar{z} = a - bi$$

FIAP

3 VETORES E ESTADOS DE UM QUBIT

Com o histórico até então, somos capazes de compreender a notação moderna dos números complexos, o que nos permite estudar suas propriedades mais a fundo.

Os qubits serão abordados com mais detalhes em capítulos futuros, em que nos aprofundaremos em sua teoria e aplicações. O estado de um qubit, em um determinado momento do sistema, é representado por uma combinação de vetores complexos. Portanto, para compreendê-lo posteriormente, precisamos relembrar as propriedades dos vetores.

3.1 O que é um vetor?

Na Física, uma grandeza que pode ser representada apenas por um valor é chamada de **grandeza escalar**, como a temperatura. Informar que um determinado ambiente está a 22 °C é suficiente para interpretar a situação.

No entanto, existem grandezas que necessitam de mais informações para serem representadas — em especial, uma direção e um sentido — e são chamadas de **grandezas vetoriais**, como as forças. Não basta dizer que há uma força de 5 *N* sendo aplicada a um corpo; é necessário informar também a direção e o sentido dessa aplicação (horizontal para a direita, por exemplo).

Algebricamente, adicionamos uma seta à variável que representa um vetor para diferenciar de um número escalar. A mesma variável, sem a seta, representa a magnitude do vetor, chamada de **módulo**.

No exemplo da força de 5 *N* aplicada a um corpo, podemos dizer:

$$\vec{F} = 5\ N \text{ na direção horizontal para a direita}$$

$$F = \left|\vec{F}\right| = 5\ N$$

Geometricamente, podemos representar um vetor como uma seta em um sistema de coordenadas. A direção e o sentido do vetor indicam para onde a seta deve apontar; seu módulo representa o comprimento do vetor.

A forma geométrica de um vetor pode ser desenhada a partir de dois pontos: uma origem e um destino. Nos estudos em Álgebra Linear, é extremamente comum

fixarmos a origem dos vetores na origem do sistema de coordenadas. Portanto, podemos descrever o vetor como apenas o ponto de destino. Esta forma de descrever o vetor é chamado de **forma cartesiana**, pois utiliza as coordenadas cartesianas.

Retomando o exemplo da força de módulo 5 *N*, como o vetor é horizontal para a direita, podemos dizer que sua origem é o ponto $(0,0)$ e seu destino é o ponto $(5,0)$. Com esta informação, é possível representar o vetor em Python utilizando o módulo NumPy.

Para isso, podemos criar uma lista que representa as coordenadas do ponto de destino e convertê-la para um *array*, como mostra o código-fonte a seguir. Este código também inclui a plotagem do gráfico com *Matplotlib*, cujo resultado está ilustrado na figura “Representação geométrica de F”.

```
import numpy as np
import matplotlib.pyplot as plt

f = np.array([5, 0)] # Resultado: [5 0]

fig, ax = plt.subplots()
ax.set_xlim(0, 7)
ax.set_ylim(0, 7)
ax.axhline(0, color='black', linewidth=0.5)
ax.axvline(0, color='black', linewidth=0.5)
ax.grid(color='gray', linestyle='--', linewidth=0,5)

ax.set_title('Vetor F')
ax.set_xlabel('x')
ax.set_ylabel('y')

ax.quiver(
    0, 0, f[0], f[1],
    angles='xy', scale_units='xy',
    scale=1, color='red'
)
```

Código-fonte 1 - Representando o vetor F em Python
Fonte: Elaborado pelo autor (2025)

Nota: de agora em diante, considere que o NumPy e Matplot sempre estarão importados. O código para gerar os gráficos também será omitido.

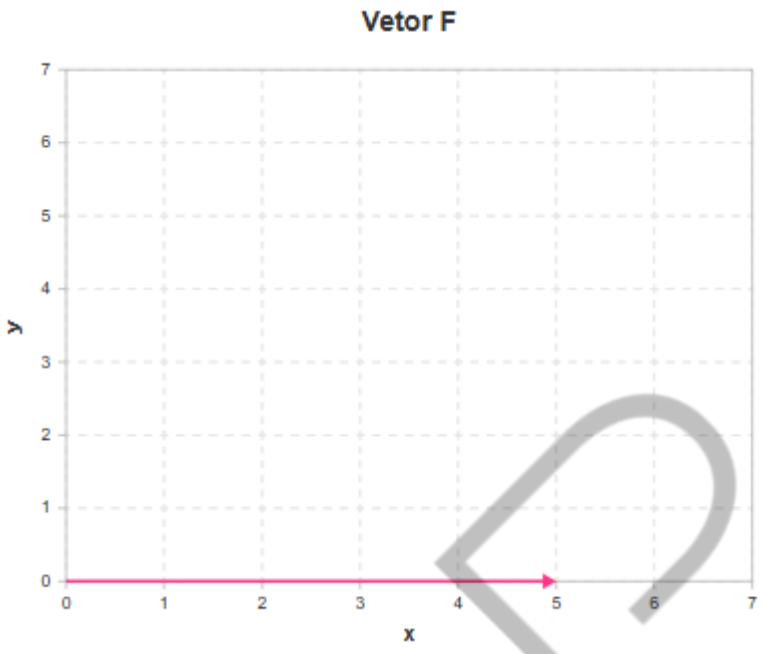


Figura 6 - Representação geométrica de F
Fonte: Elaborado pelo autor (2025)

Segundo o exemplo, seja o vetor $\vec{v} = (2,5)$. Analisando sua forma cartesiana, podemos inferir que:

- Sua origem é o ponto $(0,0)$.
- Sua extremidade é o ponto $P = (2,5)$.
- Sua direção é definida pela reta que atravessa tanto a origem quanto o ponto P.
- Seu sentido aponta da origem para o ponto P.

O código-fonte “Representando o vetor F em Python” pode ser facilmente adaptado para definir e plotar o novo vetor $\vec{v}$. A figura “Representação geométrica de v” ilustra sua representação geométrica.

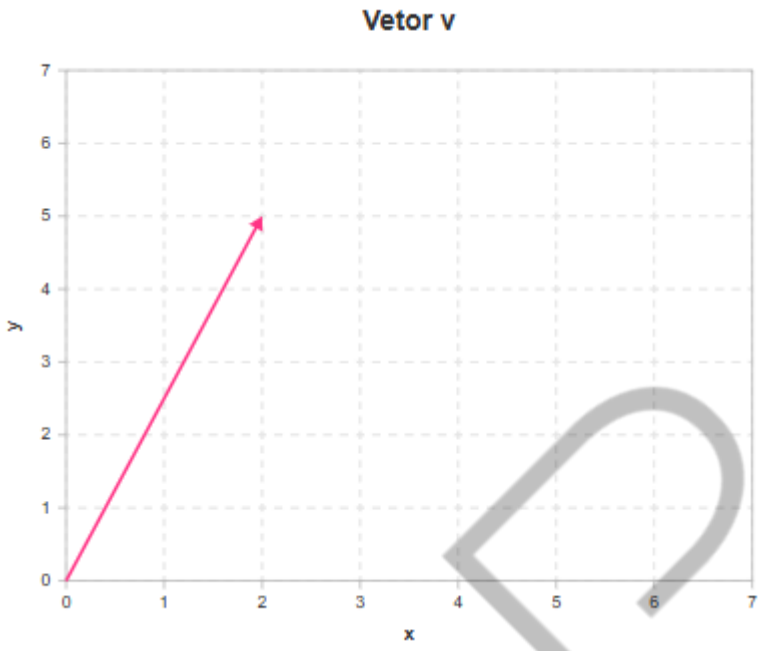


Figura 7 - Representação geométrica de v
Fonte: Elaborado pelo autor (2025)

3.2 Propriedades básicas dos vetores

Seja $\vec{u} = (5,2)$. A figura a seguir ilustra os vetores $\vec{v}$ (em rosa) e $\vec{u}$ (em azul) no mesmo plano.

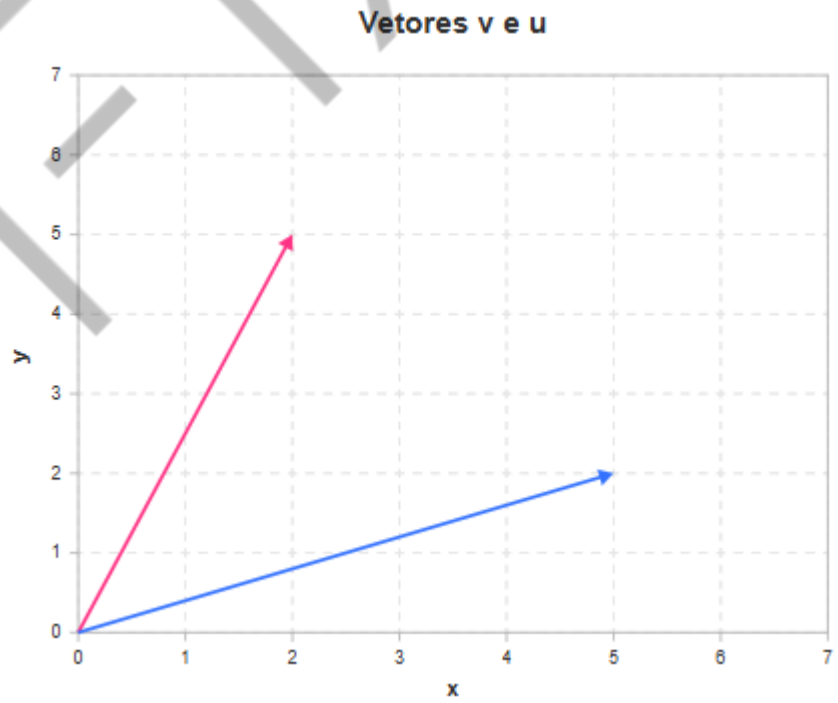


Figura 8 - Representação geométrica de v e u
Fonte: Elaborado pelo autor (2025)

A partir dos vetores de exemplo, podemos analisar outras propriedades importantes.

3.2.1 Soma e subtração de vetores

Se os vetores forem de mesma dimensão, como $\vec{v}$ e $\vec{u}$ que são ambos bidimensionais, é possível somá-los.

Algebricamente, a soma de vetores resulta em um novo vetor cujas coordenadas são a soma das coordenadas dos vetores originais.

Seja $\vec{w}$ o resultado da soma entre $\vec{v}$ e $\vec{u}$:

$$\vec{w} = \vec{v} + \vec{u}$$

$$\vec{w} = (2,5) + (5,2) = (2 + 5,5 + 2)$$

$$\vec{w} = (7,7)$$

Analogamente, podemos subtrair vetores ao subtrair suas coordenadas. Seguindo a convenção definida por Descartes, uma coordenada negativa é apenas uma distância na direção contrária ao eixo de referência.

Seja $\vec{a}$ a subtração entre $\vec{v}$ e $\vec{u}$:

$$\vec{a} = \vec{v} - \vec{u}$$

$$\vec{a} = (2,5) - (5,2) = (2 - 5,5 - 2)$$

$$\vec{a} = (-3,3)$$

Em Python, podemos somar os vetores com o operador “+” ou o método *add* do NumPy. Para subtraí-los, basta utilizar o operador “-” ou o método *subtract*, como mostra o código-fonte “Soma e subtração de vetores com NumPy”.

Por fim, a figura “Representação geométrica de v, u, w e a” mostra os vetores $\vec{v}$ (em rosa), $\vec{u}$ (em azul) e os novos $\vec{w}$ (em verde) e $\vec{a}$ (em amarelo).

```
v = np.array([2, 5])
u = np.array([5, 2])

# Método 1:
w = np.add(v, u)       # Resultado: [7 7]
a = np.subtract(v, u)  # Resultado: [-3 3]

# Método 2:
w = v + u              # Resultado: [7 7]
a = v - u              # Resultado: [-3 3]
```

Código-fonte 2 - Soma e subtração de vetores com NumPy
Fonte: Elaborado pelo autor (2025)

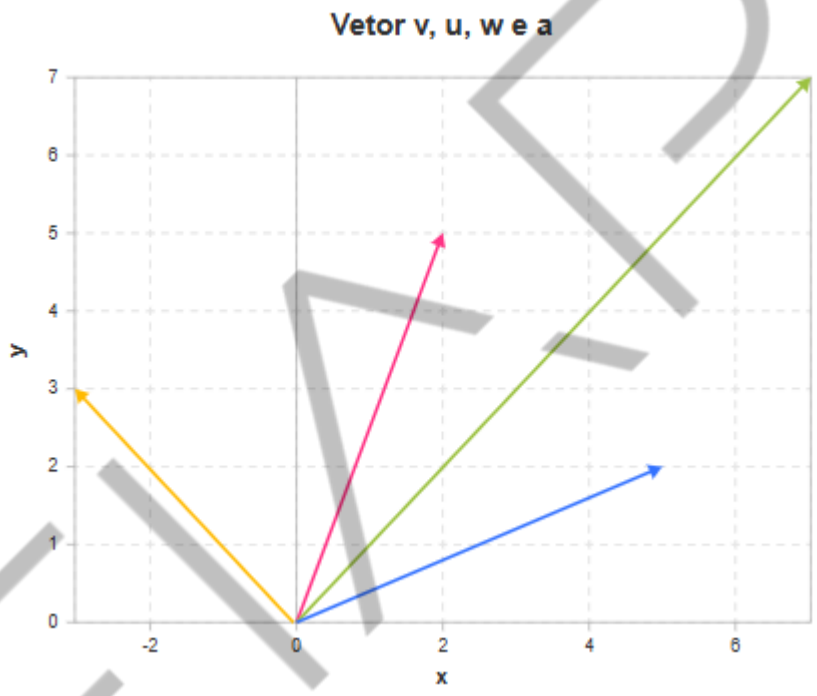


Figura 9 - Representação geométrica de v, u, w e a
Fonte: Elaborado pelo autor (2025)

Perceba que, para a soma, a ordem dos vetores é indiferente (comutatividade). Porém, para a subtração:

$$\vec{v} - \vec{u} \neq \vec{u} - \vec{v}$$

$$(2,5) - (5,2) \neq (5,2) - (2,5)$$

$$(-3,3) \neq (3,-3)$$

Importante: estas operações só podem ser efetuadas entre vetores. Não é possível somar ou subtrair uma grandeza escalar e uma vetorial.

3.2.2 Decomposição de vetores

Ao somar vetores, estamos criando um novo vetor que representa a soma entre eles. Podemos utilizar uma propriedade similar para separar um vetor em múltiplas partes.

No contexto da Computação Quântica, é comum decompormos o vetor em uma soma de suas componentes paralelas aos eixos do sistema de coordenadas. O estado de um qubit é representado por esta soma.

Essa decomposição facilita a análise e a manipulação do vetor original, permitindo que trabalhemos com seus componentes independentemente das operações e cálculos. Assim como na adição e na subtração, este raciocínio se aplica a vetores de qualquer dimensão.

Se conhecermos o ponto de destino de um vetor, sua decomposição é tão simples quanto escrever uma soma que faça com que cada uma das parcelas da soma contenha apenas uma das coordenadas do ponto de destino.

No caso de vetores bidimensionais, criaremos um vetor que só possui coordenada em x (e 0 em y) e outro que só tem coordenada em y (sendo seu x 0).

Seja $\vec{v} = (7,7)$. Podemos decompô-lo em:

$$\overrightarrow{v_x} = (7,0), \quad \overrightarrow{v_y} = (0,7)$$

De forma que:

$$\overrightarrow{v_x} + \overrightarrow{v_y} = \vec{v}$$

Não existe um método em Python ou NumPy que faça essa decomposição automaticamente, porém, este é um conceito simples que pode ser feito manualmente. O código-fonte a seguir mostra duas formas de criar os vetores $\overrightarrow{v_x}$ e $\overrightarrow{v_y}$, enquanto a figura “Representação geométrica de v, vx e vy” mostra sua representação gráfica, com $\overrightarrow{v_x}$ em azul e $\overrightarrow{v_y}$ em verde.

```
v = np.array([7, 7])

# Forma 1:
vx = np.array([v[0], 0])  # Resultado: [7 0]
vy = np.array([0, v[1]])  # Resultado: [0 7]

# Forma 2:
n_dim = v.shape[0]  # Determina o número de dimensões
components = []
for i in range(n_dim):
    component = np.zeros_like(v)
    component[i] = v[i]
    components.append(component)

vx, vy = components  # Resultados: [7 0], [0 7]
```

Código-fonte 3 - Decomposição de vetores em Python
Fonte: Elaborado pelo autor (2025)

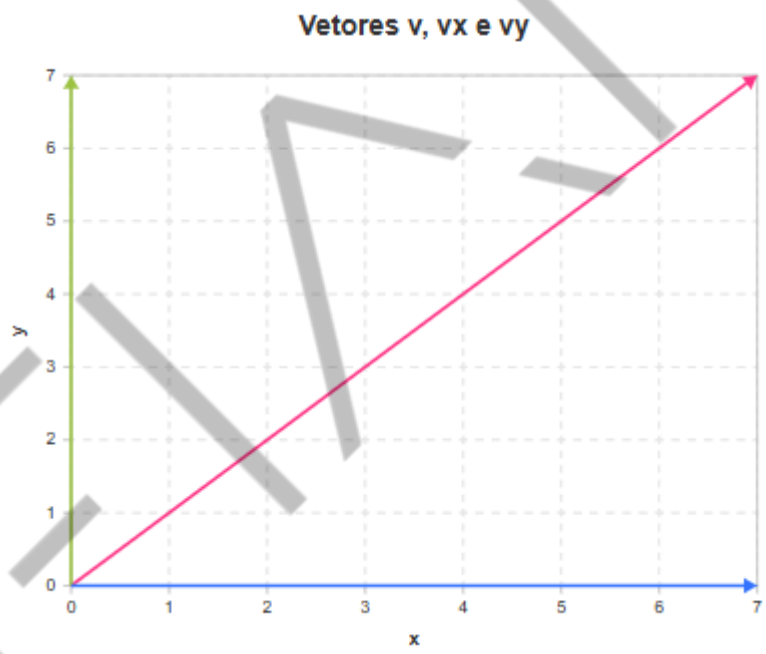


Figura 10 - Representação geométrica de v, vx e vy
Fonte: Elaborado pelo autor (2025)

3.2.3 Módulo do vetor

O módulo, também conhecido como norma ou magnitude, é o tamanho do vetor. Para indicar a norma do vetor $\vec{v}$, podemos utilizar a notação $|\vec{v}|$ ou apenas v.

A norma é calculada decompondo o vetor em suas componentes paralelas aos eixos. Em um vetor bidimensional, esta decomposição forma um triângulo retângulo de lados $\vec{v}$, $\overrightarrow{v_x}$ e $\overrightarrow{v_y}$.

Como suas componentes só possuem coordenada apenas em um eixo, seu módulo é determinado por ele. Aplicando um Pitágoras simples, podemos calcular o módulo de $\vec{v}$:

$$|\vec{v}| = \sqrt{{v_x}^2 + {v_y}^2}$$

Uma fórmula similar pode ser aplicada para qualquer vetor N-dimensional, conforme mostra a equação a seguir.

Equação 4:

$$|\vec{v}| = \sqrt{\sum_n {v_n}^2} = \sqrt{{v_x}^2 + {v_y}^2 + {v_z}^2 + \ldots}$$

Uma das aplicações da norma de um vetor na Computação Quântica é determinar se o estado de um qubit é válido.

Retornando ao exemplo $\vec{v} = (2{,}5)$, temos que:

$$\overrightarrow{v_x} = (2{,}0) \rightarrow v_x = 2$$

$$\overrightarrow{v_y} = (0{,}5) \rightarrow v_y = 5$$

$$|\vec{v}| = \sqrt{{v_x}^2 + {v_y}^2} = \sqrt{2^2 + 5^2}$$

$$|\vec{v}| = \sqrt{29} \approx 5{,}3852$$

Para calcular o módulo de um vetor em código, podemos utilizar o método **norm** do submódulo de álgebra linear do NumPy, como mostra o código-fonte:

```
v = np.array([2, 5])

norma = np.linalg.norm(v) # Resultado: 5.385164807134504
```

Código-fonte 4 - Calculando a norma de um vetor em Python
Fonte: Elaborado pelo autor (2025)

Vale destacar que, para vetores complexos, é possível calcular seu módulo multiplicando o vetor por seu conjugado:

$$z\bar{z} = \bar{z}z = |z|^2 = |\bar{z}|^2 \leftrightarrow |z| = \sqrt{z\bar{z}}$$

3.2.4 Multiplicação de vetor por escalar

Ao contrário do que acontece com a adição e a subtração, vetores podem ser multiplicados por uma grandeza escalar.

Geometricamente, multiplicação por escalar aplica uma transformação no vetor, que:

- Aumenta, caso o escalar seja maior que 1.
- Diminui, caso o escalar seja um número entre 0 e 1.
- Mantém sua direção, mas tem seu sentido invertido, caso o escalar seja negativo.

Algebricamente, apenas multiplicamos todos os elementos do vetor pelo escalar:

$$n \cdot \vec{v} = (n \cdot v_x, n \cdot v_y, n \cdot v_z, \dots)$$

Como exemplo, considere estes quatro vetores:

$$\vec{v} = (2,5)$$

$$\vec{u} = 2 \cdot \vec{v} = (4,10)$$

$$\vec{w} = \frac{1}{2}\vec{v} = \left(1, \frac{5}{2}\right)$$

$$\vec{a} = -1 \cdot \vec{w} = \left(-1, -\frac{5}{2}\right)$$

Em Python, podemos multiplicar um vetor por escalar utilizando o operador “*” ou o método *multiply* do NumPy, como mostra o código-fonte a seguir. A figura “Representação geométrica dos múltiplos de v” ilustra os vetores $\vec{v}$, $\vec{u}$, $\vec{w}$ e $\vec{a}$, respectivamente em rosa, azul, verde e amarelo.

```
v = np.array([2, 5])

# Método 1
u = np.multiply(2, v)     # Resultado: [4 10]
w = np.multiply(0.5, v)   # Resultado: [1 2.5]
a = np.multiply(-1, w)    # Resultado: [-1 -2.5]

# Método 2
u = 2 * v                 # Resultado: [4 10]
w = 0.5 * v               # Resultado: [1 2.5]
a = -1 * v                # Resultado: [-1 -2.5]
```

Código-fonte 5 - Exemplos de multiplicação de vetor por escalar em Python
Fonte: Elaborado pelo autor (2025)

Observe que a recíproca é verdadeira: se todos os elementos da matriz forem múltiplos de um certo n escalar, podemos “colocá-lo em evidência”, representando a matriz como uma multiplicação de n pela nova matriz fatorada (todos os elementos divididos por n).

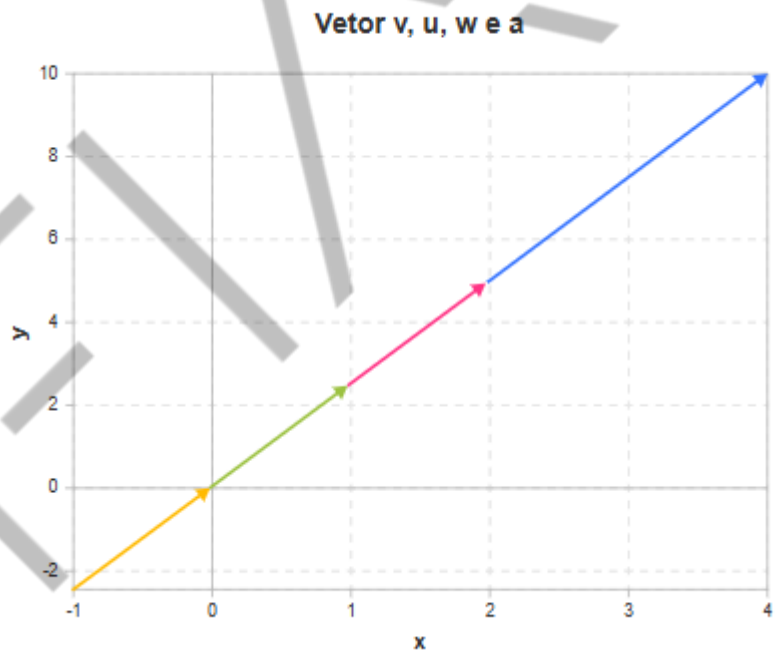


Figura 11 - Representação geométrica dos múltiplos de v
Fonte: Elaborado pelo autor (2025)

3.3 Multiplicação entre vetores

Existem diversos tipos diferentes de multiplicação entre vetores, cada um com suas propriedades e interpretações físicas. Para a Computação Quântica, os mais

úteis são o **produto escalar**, conhecido como **produto interno**, e **o produto vetorial**, também chamado de **produto externo**.

3.3.1 Produto Interno

Este produto relaciona um par de vetores a um número escalar. Sua interpretação depende do contexto: se os vetores forem forças (*A* e *B*), o produto interno pode representar o quanto da força *A* está sendo aplicada em *B*.

No contexto da Computação Quântica, em que os vetores são estados de qubits, seu produto interno representa a **amplitude de probabilidade** de um sistema — isto é, a chance inicial de um dos estados ser encontrado no outro estado após uma medição. Assim, o produto interno é a base para medirmos qubits em um circuito quântico.

O produto interno pode ser calculado de duas formas diferentes. Se tivermos a representação gráfica dos dois vetores, conhecendo seus módulos e o ângulo θ entre eles, podemos utilizar a seguinte relação:

$$\vec{v} \cdot \vec{u} = |\vec{v}| \cdot |\vec{u}| \cdot cos\ \theta$$

Com base nas leis da Trigonometria, definimos que $v \cdot cos\ \theta$ é a componente decomposta de $\vec{v}$. Dessa forma, o produto interno é a **projeção** de $\vec{v}$ na direção de $\vec{u}$, como se fosse a “sombra” que um vetor faz sobre o outro.

Outra forma, que tende a ser mais útil no contexto em questão, é utilizar as coordenadas do ponto de destino de cada vetor para calcular o produto. Neste caso, cada elemento do vetor é multiplicado pelo correspondente no outro vetor. Por fim, somamos os resultados. A equação a seguir mostra a fórmula geral, para vetores N-dimensionais.

Equação 5:

$$\vec{v} \cdot \vec{u} = \sum_{i} \quad v_i \cdot u_i$$

Essa forma de se escrever o produto destaca a necessidade de termos vetores de mesma dimensão.

Importante: ao contrário da multiplicação clássica entre números escalares, que aceita várias notações diferentes, o produto interno **deve** ser escrito como "·", o que dá um de seus nomes em inglês: "*dot product*" ("produto em ponto").

Para exemplificar, seja $\vec{v} = (2,6,1)$ e $\vec{u} = (5,3,4)$.

$$\vec{v} \cdot \vec{u} = (2,6,1) \cdot (5,3,4)$$

$$\vec{v} \cdot \vec{u} = 2 \cdot 5 + 6 \cdot 3 + 1 \cdot 4$$

$$\vec{v} \cdot \vec{u} = 10 + 18 + 4 = 32$$

Em Python, podemos calcular o produto interno utilizando o método *dot* do NumPy, como mostra o código-fonte:

```
v = np.array([2, 6, 1])
u = np.array([5, 3, 4])

p_interno = np.dot(v, u) # Resultado: 32
```

Código-fonte 6 - Exemplo de produto interno em Python
Fonte: Elaborado pelo autor (2025)

Entre as propriedades do produto interno, podemos destacar:

- Pode ser utilizado para calcular a norma do vetor: $|\vec{v}| = \sqrt{\vec{v} \cdot \vec{v}}$.
- Comutativa: $\vec{v} \cdot \vec{u} = \vec{u} \cdot \vec{v}$.
- Distributiva: $\vec{a} \cdot (\vec{v} + \vec{u}) = \vec{a} \cdot \vec{v} + \vec{a} \cdot \vec{u}$

3.3.2 Produto externo

Graficamente, esta operação retorna um novo vetor, perpendicular ao plano formado pelos vetores multiplicados. Algumas de suas principais aplicações são o cálculo de áreas de triângulos e a identificação da direção relativa entre vetores. Para a Computação Quântica, o produto externo aparece recorrentemente em diversos cálculos, além de ser a base para criarmos um operador quântico.

Se conhecermos a norma dos vetores e um vetor de módulo 1 perpendicular ao plano de $\vec{v}$ e $\vec{u}$ (versor $\hat{n}$), podemos calcular o produto externo como:

$$\vec{v} \times \vec{u} = \hat{n} \cdot v \cdot u \cdot sin\,\theta$$

Importante: assim como no produto interno, o produto externo tem sua notação própria e **deve** ser escrito como "×", o que dá um de seus nomes em inglês: "*cross product*" ("produto em cruz").

A outra forma de se calcular o produto externo depende da notação de matrizes, que veremos a seguir. O código-fonte "Exemplo de produto externo em Python" mostra como calcular o produto vetorial em Python, utilizando o método *outer* do NumPy.

```
v = np.array([2, 6, 1])
u = np.array([5, 3, 4])

p_externo = np.outer(v, u)
# Resultado:
# [[10  6  8]
#  [30 18 24]
#  [ 5  3  4]]
```

Código-fonte 7 - Exemplo de produto externo em Python
Fonte: Elaborado pelo autor (2025)

Entre as propriedades do produto interno, podemos destacar:

- Anti-comutatividade: $\vec{v} \times \vec{u} = -\vec{u} \times \vec{v}$
- $\vec{v} \times \vec{u} \neq \vec{u} \times \vec{v}$
- Distributiva **apenas** na adição: $\vec{a} \times (\vec{v} + \vec{u}) = \vec{a} \times \vec{v} + \vec{a} \times \vec{u}$

4 MATRIZES E OPERADORES QUÂNTICOS

Matrizes são dados organizados em linhas e colunas, similar a uma tabela. Na computação, podemos representar como uma lista ou *array* cujos elementos são um conjunto de listas. Podem ser utilizadas em diversos contextos, como manipular vetores, uma propriedade importante no contexto da Computação Quântica.

Uma matriz é definida por uma letra e seu número de linhas (n) e colunas (m), respectivamente: $A_{n\times m}$. Por exemplo, a matriz $A_{2\times 3}$ possui duas linhas e três colunas:

$$A_{2\times 3} = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{pmatrix}$$

Seus elementos são denominados por a_{ij}, em que i é o índice da linha e j é o índice da coluna.

- A matriz é denominada de “matriz quadrada” caso tenha o mesmo número de linhas e colunas.
- Caso tenha apenas uma linha, é chamada de “matriz linha”.
- Caso tenha apenas uma coluna, é chamada de “matriz coluna”.

4.1 Propriedades básicas

As matrizes possuem várias propriedades em comum com os vetores, nos permitindo, inclusive, representar vetores como matrizes linha ou coluna.

$$\vec{v} = \begin{pmatrix} v_x & v_y & v_z \end{pmatrix} \qquad \vec{v} = \begin{pmatrix} v_x \\ v_y \\ v_z \end{pmatrix}$$

Algumas das propriedades que podemos destacar são a soma de matrizes e a multiplicação por escalar, análogas às operações entre vetores (note que, para a soma, as matrizes devem ter as mesmas dimensões).

$$C_{2\times3} = A_{2\times3} + B_{2\times3} = \begin{pmatrix} (a_{11} + b_{11}) & (a_{12} + b_{12}) & (a_{13} + b_{13}) \\ (a_{21} + b_{21}) & (a_{22} + b_{22}) & (a_{23} + b_{23}) \end{pmatrix}$$

$$n \cdot C = \begin{pmatrix} n \cdot C_{11} & n \cdot C_{12} & n \cdot C_{13} \\ n \cdot C_{21} & n \cdot C_{22} & n \cdot C_{23} \end{pmatrix}$$

O código-fonte “Exemplo de soma de matrizes e multiplicação por escalar em Python” exemplifica a criação de duas matrizes quadradas 3×3, sua soma e uma multiplicação por valor escalar. Uma matriz é um *array* cujos elementos são, por sua vez, outros *arrays*: cada *subarray* representa uma linha da matriz. Como foi o caso com vetores, podemos utilizar os operadores aritméticos (“+” e “*”) ou os mesmos métodos (*add* e *multiply*).

```
a = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
# [[1 2 3]
#  [4 5 6]
#  [7 8 9]]

b = np.array([[9, 8, 7], [6, 5, 4], [3, 2, 1]])
# [[9 8 7]
#  [6 5 4]
#  [3 2 1]]

c = a + b
# [[10 10 10]
#  [10 10 10]
#  [10 10 10]]

c_vezes_dois = 2 * c
# Resultado:
# [[20 20 20]
#  [20 20 20]
#  [20 20 20]]
```

Código-fonte 8 - Exemplo de soma de matrizes e multiplicação por escalar em Python
Fonte: Elaborado pelo autor (2025)

Assim como acontece com vetores, podemos subtrair uma matriz de outra ao subtrair seus elementos. Em código, utilizamos os mesmos operadores e métodos.

4.2 Multiplicação de matrizes

4.2.1 Produto matricial

A multiplicação de matrizes pode ser utilizada para realizar uma transformação em um vetor. No contexto da Computação Quântica, isso pode significar a aplicação de um operador, ou porta lógica, sobre um qubit.

Seu algoritmo é diferente do usual. Primeiramente, só é possível multiplicar matrizes caso o número de **colunas** da primeira seja igual ao número de **linhas** da segunda, resultando em uma matriz com o número de linhas da primeira e o número de colunas da segunda. Vale destacar que a multiplicação não é comutativa:

$$C_{n\times m} = A_{n\times k} \cdot B_{k\times m}$$

$$AB \neq BA$$

Se a condição for cumprida, cada elemento c_{ij} de C é calculado como a soma entre a multiplicação dos elementos da linha i de A e a coluna j de B:

$$c_{ij} = \sum_{k} \quad a_{ik}b_{kj}$$

Por exemplo, seja $C_{2\times 2} = A_{2\times 3}B_{3\times 2}$:

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{pmatrix} \qquad B = \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \\ b_{31} & b_{32} \end{pmatrix}$$

$$C = \begin{pmatrix} (a_{11} \cdot b_{11} + a_{12} \cdot b_{21} + a_{13} \cdot b_{31}) & (a_{11} \cdot b_{12} + a_{12} \cdot b_{22} + a_{13} \cdot b_{32}) \\ (a_{21} \cdot b_{11} + a_{22} \cdot b_{21} + a_{23} \cdot b_{31}) & (a_{21} \cdot b_{12} + a_{22} \cdot b_{22} + a_{23} \cdot b_{32}) \end{pmatrix}$$

Para realizar a operação em Python, podemos utilizar o método *dot* do NumPy, tal qual o produto interno de vetores, ou o operador “@”, como mostra o código-fonte “Exemplo de multiplicação entre matrizes em Python”.

```
a = np.array([[1, 2, 3], [4, 5, 6]])
b = np.array([[9, 8], [6, 5], [3, 2]])

c = np.dot(a, b)
c = a @ b
# [[30 24]
#  [84 69]]
```

Código-fonte 9 - Exemplo de multiplicação entre matrizes em Python
Fonte: Elaborado pelo autor (2025)

4.2.2 Produto tensorial

Além da multiplicação clássica, temos o produto tensorial, que é utilizado para definir tensores e para construir um novo espaço vetorial — conjunto de vetores que podem ser somados ou multiplicados por escalares, representando entidades físicas.

Para a Computação Quântica, utilizamos o produto tensorial para criar um Espaço de Hilbert — espaço vetorial no conjunto dos Complexos que também define produto interno —, onde os estados de um sistema quântico são descritos, além de definir operadores lógicos aplicados em múltiplos qubits.

O produto tensorial entre dois vetores retorna uma matriz cujos elementos são o produto de um elemento do primeiro vetor com cada um dos elementos do segundo:

$$\vec{v} = \begin{pmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{pmatrix} \quad \vec{w} = \begin{pmatrix} w_1 \\ w_2 \\ w_3 \end{pmatrix}$$

$$\vec{v} \otimes \vec{w} = \begin{pmatrix} v_1 w_1 & v_1 w_2 & v_1 w_3 \\ v_2 w_1 & v_2 w_2 & v_2 w_3 \\ v_3 w_1 & v_3 w_2 & v_3 w_3 \\ v_4 w_1 & v_4 w_2 & v_4 w_3 \end{pmatrix}$$

É fácil ver que o produto tensorial não é comutativo, porém, os vetores não precisam ter as mesmas dimensões. Note, ainda, que esta operação também possui um símbolo específico, “⊗”.

Em Python, é possível calcular o produto tensorial utilizando o método mais geral *tensordot* do NumPy (definindo o eixo como 0), ou o método *kron* caso a multiplicação seja entre *arrays* de uma ou duas dimensões. Estas operações estão exemplificadas no código-fonte:

```
a = np.array([10, 20])
b = np.array([3, 4, 5])

c = np.tensordot(a, b, axes=0)
# [[30 40  50]
#  [60 80 100]]
```

Código-fonte 10 - Exemplo de produto tensorial em Python
Fonte: Elaborado pelo autor (2025)

4.3 Transposta

Apesar de ser um conceito simples e sem significado físico direto, transpor uma matriz é um conceito importante que será utilizado em diversas operações de agora em diante.

Transpor uma matriz significa criar uma nova matriz invertendo o eixo dos elementos da matriz original, ou seja:

$$(A_{n\times m})^T = A_{m\times n}$$

$$a_{ij} \rightarrow a_{ji}$$

A transposta é facilmente determinada em Python utilizando o método *T* do tipo *array* NumPy, conforme ilustra o código-fonte:

```
m = np.array([[1, 2, 3],
              [4, 5, 6]])

m_t = m.T
# [[1 4]
#  [2 5]
#  [3 6]]
```

Código-fonte 11 - Exemplo de transposta de matriz em Python
Fonte: Elaborado pelo autor (2025)

4.4 Matriz Identidade

A Identidade é o elemento neutro da multiplicação de matrizes, de forma que, para qualquer matriz A:

$$A \cdot I = I \cdot A = A$$

Para tal, a Identidade é uma matriz quadrada cujos elementos da diagonal principal (a_{ij} com $i = j$) são 1 e todos os outros são 0, podendo ter qualquer tamanho, como nos exemplos:

$$\mathbb{I}_2 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \quad \mathbb{I}_3 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad \mathbb{I}_4 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

4.5 Matrizes complexas

Ao trabalhar com matrizes que contém elementos complexos, é importante destacar algumas propriedades fundamentais para a futura definição dos operadores lógicos em circuitos quânticos.

4.5.1 Conjugado e Transposta Hermitiana

Para aplicar o conjugado a uma matriz, basta conjugar todos os seus elementos. Por exemplo:

$$Z = \begin{pmatrix} 1 & -i \\ 5i & 4 - 21 \end{pmatrix} \leftrightarrow Z^C = \begin{pmatrix} 1^C & (-i)^C \\ (5i)^C & 4 - 21^C \end{pmatrix} = \begin{pmatrix} 1 & i \\ -5i & 4 + 21 \end{pmatrix}$$

Com isso, podemos definir a operação Transposta Hermitiana, que consiste em calcular o conjugado da transposta da matriz: $(Z^T)^C$. Também podemos usar o símbolo "†" para representar a operação, que recebe o apelido de "*dagger*".

$$Z^T = \begin{pmatrix} 1 & 5i \\ -i & 4 - 21 \end{pmatrix} \leftrightarrow (Z^T)^C = \begin{pmatrix} 1 & -5i \\ i & 4 + 21 \end{pmatrix} = Z^\dagger$$

É possível também aplicar o *dagger* em vetores, sendo equivalente ao conjugado:

$$zz^\dagger = |\bar{z}|^2$$

4.5.2 Matriz unitária

Uma matriz unitária "U" é uma matriz quadrada que satisfaz a relação:

$$UU^\dagger = U^\dagger U = I$$

Todas as portas lógicas quânticas devem ser matrizes unitárias para serem válidas. Por exemplo, considere a matriz A:

$$A = \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{i}{\sqrt{2}} & -\frac{i}{\sqrt{2}} \end{pmatrix} = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ i & -i \end{pmatrix} \rightarrow A^\dagger = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & -i \\ 1 & i \end{pmatrix}$$

$$AA^\dagger = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ i & -i \end{pmatrix}\frac{1}{\sqrt{2}}\begin{pmatrix} 1 & -i \\ 1 & i \end{pmatrix} = \frac{1}{2}\begin{pmatrix} 1+1 & (-i)+i \\ i-i & i\cdot(-i)+(-i)\cdot i \end{pmatrix} = \frac{1}{2}\begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

$$\therefore AA^\dagger = \mathbb{I}$$

Utilizando um passo a passo similar, é fácil ver que $A^\dagger A$ também resulta na Identidade, ou seja, a matriz A é um operador quântico válido que pode agir sobre um qubit.

4.6 Notação de Dirac

Na Computação Quântica, é extremamente comum escrevermos vetores como matrizes linha ou coluna. Se aproveitando das propriedades de decomposição de vetores e sua combinação linear, Paul Dirac criou uma notação simplificada para tais vetores que representam o estado de um qubit, chamada de "*bra-ket*" ou "notação de Dirac".

Um "*bra*" é um vetor linha contendo um elemento igual a 1 e o restante igual a 0. Ele é representado por $\langle i|$ (lê-se "*bra* i"), em que i é o índice do elemento 1 no vetor, não importando sua dimensão:

$$\vec{v} = (1\ 0) = \langle 0|$$

$$\vec{v} = (1\ 0\ 0) = \langle 0|$$

$$\vec{v} = (0\ 1\ 0) = \langle 1|$$

Já o "*ket*", representado por $|i\rangle$ ("*ket*" i), é um vetor coluna com as mesmas propriedades.

$$\vec{v} = \begin{pmatrix}1\\0\end{pmatrix} = |0\rangle, \quad \vec{v} = \begin{pmatrix}1\\0\\0\end{pmatrix} = |0\rangle, \quad \vec{v} = \begin{pmatrix}0\\0\\1\\0\end{pmatrix} = |2\rangle$$

Vale destacar que *bra* e *ket* são o *dagger* um do outro:

$$\langle i|^\dagger = |i\rangle \leftrightarrow |i\rangle^\dagger = \langle i|$$

4.6.1 Composição linear

Caso o vetor tenha valores diferentes de 0 e 1, ou mais de um valor que não seja nulo, podemos decompô-lo e representá-lo como uma soma:

$$\vec{v} = \begin{pmatrix}1\\2\\3\end{pmatrix} = \begin{pmatrix}1\\0\\0\end{pmatrix} + \begin{pmatrix}0\\2\\0\end{pmatrix} + \begin{pmatrix}0\\0\\3\end{pmatrix}$$

$$\vec{v} = \begin{pmatrix}1\\0\\0\end{pmatrix} + 2\begin{pmatrix}0\\1\\0\end{pmatrix} + 3\begin{pmatrix}0\\0\\1\end{pmatrix} = |0\rangle + 2|1\rangle + 3|2\rangle$$

4.6.2 Simplificando Produto Interno e Externo

É fácil perceber que, ao multiplicarmos uma matriz linha por uma matriz coluna com a mesma quantidade de elementos, realizamos uma operação análoga ao produto interno de dois vetores.

Assim, ao utilizar a notação de Dirac, podemos simplificar os cálculos de produto interno entre vetores ao escrevê-los em forma de *braket*:

$$\vec{v} \cdot \vec{u} = \langle v| \cdot \langle u|^\dagger = \langle v| \cdot |u\rangle = \langle v|u\rangle$$

Como o produto interno multiplica valores de mesmo índice, e vetores na notação de Dirac só possuem valores 0 e 1, concluímos que se:

- Os vetores forem iguais, seu produto será 1.
- Os vetores forem diferentes, seu produto será 0.

Esta propriedade será utilizada para determinar a ortogonalidade dos vetores.

De maneira semelhante, a notação de Dirac simplifica o cálculo do produto externo, que sempre resultará em uma matriz com um elemento igual a 1 e o restante nulo.

O primeiro vetor indica o índice da linha do elemento não nulo (começando a contagem por 0), enquanto o segundo vetor indica o índice de sua coluna:

$$\vec{v} \times \vec{u} = |v\rangle \times \langle u| = |v\rangle \langle u|$$

$$|0\rangle \langle 1| = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \begin{pmatrix} 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 \cdot 0 & 1 \cdot 1 \\ 0 \cdot 0 & 0 \cdot 1 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$$

4.7 Autovetores e autovalores

Multiplicar uma matriz por um vetor o transforma, podendo alterar seu módulo ou realizar uma rotação. Se uma matriz A efetuar uma transformação em $\vec{v}$, podemos encontrar um escalar λ que efetuaria a mesma transformação, como mostra a equação.

Equação 6:

$$A\vec{v} = \lambda\vec{v}$$

O vetor $\vec{v}$ é chamado de **autovetor** — um vetor que, ao ser transformado pela matriz, resulta em um múltiplo de si mesmo. Já λ é o **autovalor**: o escalar que, multiplicado pelo autovetor, resulta no mesmo vetor transformado por A.

Na Computação Quântica, os estados dos qubits são transformados pelos operadores, tornando-os autovetores. Seus autovalores correspondem aos possíveis resultados de medição do sistema. Ao medir, o sistema colapsa para o estado do autovetor correspondente ao autovalor calculado.

O submódulo de Álgebra Linear do NumPy nos fornece métodos simples para calcular os autovetores (“*eigenvectors*”) e os autovalores (“*eigenvalues*”), como mostra o código-fonte a seguir.

```
A = np.array([[2, 1], [1, 2]])
autovalores, autovetores = np.linalg.eig(A)

print(autovalores) # [3. 1.]
print(autovetores)
# [[ 0.70710678 -0.70710678]
#  [ 0.70710678  0.70710678]]
```

Código-fonte 12 - Calculando autovetores e autovalores em Python
Fonte: Elaborado pelo autor (2025)

O resultado do código-fonte nos informa uma lista de autovalores e uma matriz correspondente aos autovetores. Cada **coluna** da matriz se relaciona com o autovalor de mesmo índice.

Para confirmar o resultado, basta escolher um autovalor com seu vetor correspondente e verificar a igualdade contida na **equação 6**.

FIAP

5 CRIANDO E MEDINDO ESTADOS DE QUBITS

O estado de um qubit é representado por um vetor complexo, normalmente escrito utilizando a notação de Dirac. Para que o estado seja válido, **a norma** deste vetor deve ser igual a 1, do contrário, o vetor não pode ser um estado.

Por exemplo:

$$\vec{v} = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix} = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

$$|\vec{v}| = \sqrt{\left|\frac{1}{\sqrt{2}}\right|^2 + \left|\frac{1}{\sqrt{2}}\right|^2} = \sqrt{\frac{1}{2} + \frac{1}{2}} = 1$$

Indicando que este é um estado válido, diferente de:

$$\vec{v} = \begin{pmatrix} \frac{1}{2} \\ \frac{1}{2} \end{pmatrix} = \frac{1}{2}(|0\rangle + |1\rangle)$$

$$|\vec{v}| = \sqrt{\left|\frac{1}{2}\right|^2 + \left|\frac{1}{2}\right|^2} = \sqrt{\frac{1}{4} + \frac{1}{4}} = \frac{1}{\sqrt{2}} \neq 1$$

Em código, além do NumPy, podemos utilizar a biblioteca Qiskit, criada pela IBM para criar circuitos quânticos e algoritmos, sendo a mais popular hoje em dia.

Após instalar o pacote *qiskit* com o pip, podemos instanciar um objeto da classe *Statevector*, que cria um vetor de estado a partir de uma lista ou um *array* do NumPy. O código-fonte “Criando vetores de estado em Qiskit” mostra um exemplo de *Statevector* e alguns de seus métodos básicos.

```
from qiskit.quantum_info import Statevector

ket0 = Statevector([1, 0])

# Print normal do vetor
print(ket0.data) # [1+0.j 0+0.j]

# Se estiver em um Jupyter, podemos desenhar o vetor
ket0.draw() # por padrão, escreve o estado em ASCII
ket0.draw('latex') # desenha o vetor em LaTeX

# Checa se o vetor é válido
print(ket0.is_valid()) # True
```

Código-fonte 13 - Criando vetores de estado em Qiskit
Fonte: Elaborado pelo autor (2025)

5.1 Modificando um estado

Como mencionado anteriormente, para manipular o estado de um qubit, utilizamos operadores quânticos, que são matrizes complexas unitárias. Por enquanto, podemos definir um operador como uma matriz do NumPy e aplicá-lo ao calcular a multiplicação normalmente. No próximo capítulo, criaremos circuitos para manipular os estados de uma forma mais eficiente.

O código-fonte "Aplicando transformações de estado em Qiskit" ilustra a criação da matriz X, que representa uma operação de negação: aplicá-la ao estado 0 o transforma em 1 e vice-versa.

```
X = np.array([[0, 1], [1, 0]])
ket0 = np.array([1, 0])

ket1 = np.dot(X, ket0)
ket1_state = Statevector(ket1)

print(ket1_state.data) # [0.+0.j 1+0.j]
print(ket1_state.is_valid()) # True
```

Código-fonte 14 - Aplicando transformações de estado em Qiskit
Fonte: Elaborado pelo autor (2025)

Este é apenas um exemplo ilustrativo, no próximo capítulo estudaremos em detalhes a criação de um operador e seus significados.

5.2 Medindo um estado

Para calcular a probabilidade de se encontrar o sistema em um determinado estado após uma medição, calcula-se o **produto interno** entre o estado cuja probabilidade queremos descobrir e o vetor de estado do qubit. A probabilidade de medição será o **módulo ao quadrado** deste produto. Por exemplo, seja o estado ψ definido a seguir; a probabilidade que uma medição resulte no estado 0 é:

$$|\psi\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

$$\langle 0|\psi\rangle = \left\langle 0\middle|\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)\right\rangle = \frac{1}{\sqrt{2}}(\langle 0|0\rangle + \langle 0|1\rangle) = \frac{1}{\sqrt{2}}(1+0) = \frac{1}{\sqrt{2}}$$

$$|\langle 0|\psi\rangle|^2 = \left|\frac{1}{\sqrt{2}}\right|^2 = \frac{1}{2} = 50\%$$

Em Qiskit, podemos fazer de duas formas: utilizar o método *inner* de *Statevector* para calcular o produto interno entre um vetor e outro, ou usar diretamente o método *probabilities*, que retorna uma lista com probabilidades associadas à medição de cada estado.

O código-fonte “Calculando probabilidades em Qiskit” ilustra o exemplo anterior utilizando Qiskit. É possível perceber os erros de arredondamento causados pelos valores decimais.

```
ket0 = np.array([1, 0])
ket_p = Statevector((1/np.sqrt(2))*np.array([1,1]))

print(ket_p) # [0.70710678+0.j 0.70710678+0.j]

# Calculando manualmente a probabilidade de medir zero
prob = np.abs(ket0.inner(ket_p))**2
print(prob) # 0.4999999999999999

# Calculando todas as probabilidades automaticamente
prob = ketp.probabilities() # [0.5 0.5]
print(prob[0]) # 0.4999999999999999
```

Código-fonte 15 - Calculando probabilidades em Qiskit
Fonte: Elaborado pelo autor (2025)

5.3 Visualização de estados

Outra forma de se representar o estado de um qubit é utilizando a Esfera de Bloch, uma ferramenta sem significado físico utilizada para visualizar o vetor de estados.

O polo norte da esfera representa o estado 0; seu polo sul representa o estado 1. Caso o vetor esteja entre os dois polos, representa um estado em sobreposição, como o estado ψ do exemplo anterior.

O código-fonte a seguir mostra como desenhar a Esfera com Qiskit para representar os estados 0, 1 e ψ. As figuras “Estado zero na Esfera de Bloch”, “Estado um na Esfera de Bloch” e “Estado em superposição na Esfera de Bloch” apresentam o resultado.

```
from qiskit.visualization import plot_bloch_multivector

ket0 = np.array([1, 0])
ket0 = np.array([0, 1])
ket_p = Statevector((1/np.sqrt(2))*np.array([1,1]))

plot_bloch_multivector(ket0, title='Estado zero')
plt.show()
plot_bloch_multivector(ket1, title='Estado um)
plt.show()
plot_bloch_multivector(ket_p, title='Estado psi)
plt.show()
```

Código-fonte 16 - Visualizando a Esfera de Bloch em Qiskit
Fonte: Elaborado pelo autor (2025)

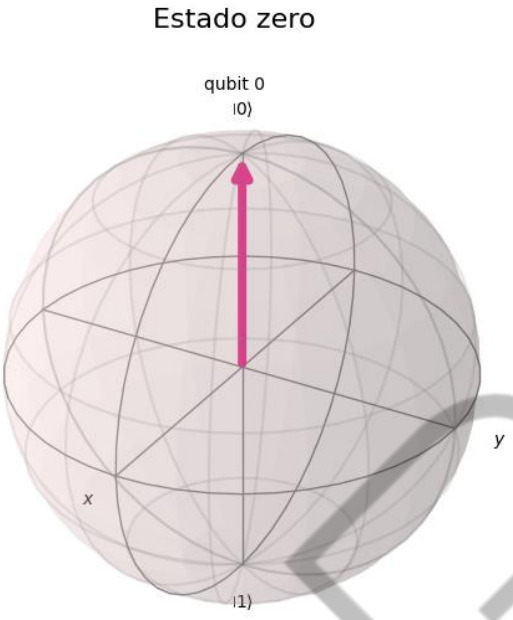


Figura 12 - Estado zero na Esfera de Bloch
Fonte: Elaborado pelo autor (2025)

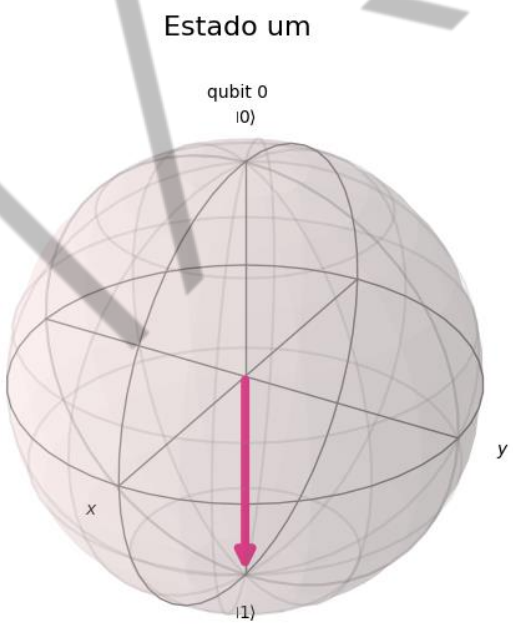


Figura 13 - Estado um na Esfera de Bloch
Fonte: Elaborado pelo autor (2025)

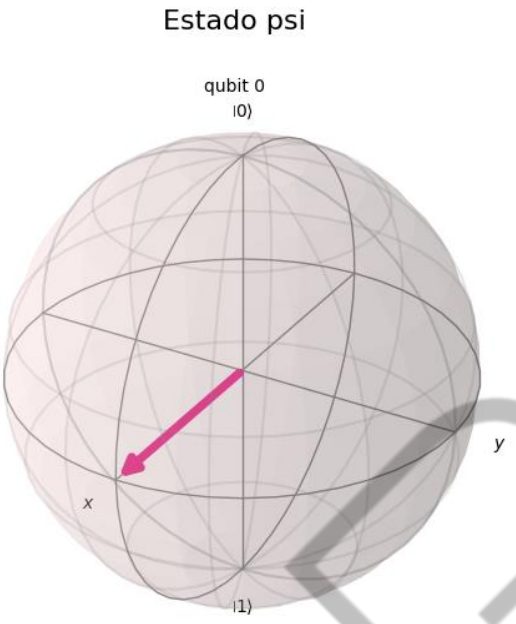


Figura 14 - Estado em superposição na Esfera de Bloch
Fonte: Elaborado pelo autor (2025)

6 CONCLUSÃO

6.1 O que aprendemos com a Matemática

Ao longo deste capítulo, discutimos os conceitos matemáticos que servem como base para a Computação Quântica. Iniciamos com a história dos números complexos, dos paradoxos da Antiguidade até a formulação moderna de Argand e Gauss, e seguimos com a apresentação de vetores e suas operações, essenciais para representar estados quânticos.

As propriedades de matrizes e os operadores quânticos foram apresentados como os meios para manipular esses estados, destacando as matrizes unitárias e o operador hermitiano. Por fim, utilizamos a notação de Dirac para expressar transformações e medições de forma mais simplificada.

Vimos na prática que a Matemática é mais do que realizar cálculos: cada elemento tem um significado real; cada vetor representa um estado possível; cada matriz, uma transformação; e cada projeção, uma medição que altera o sistema. A visualização na Esfera de Bloch reafirmou esse entendimento, mostrando que mesmo os conceitos mais abstratos podem ter forma no contexto quântico.

6.2 Da Matemática à Computação Quântica

Com a base adquirida neste capítulo, temos as ferramentas necessárias para compreender como circuitos e algoritmos quânticos operam, permitindo os primeiros passos práticos na Programação Quântica. A Matemática, longe de ser um pré-requisito meramente técnico, revela-se como o próprio tecido sobre o qual a realidade quântica é modelada e explorada.

No próximo capítulo, exploraremos os fundamentos da Computação Quântica e dos qubits, aplicando diretamente os conceitos matemáticos aqui estudados e aprofundando em seus significados físicos.

Serão introduzidos os estados básicos $|0\rangle$ e $|1\rangle$, suas representações no espaço vetorial e visualização na Esfera de Bloch. Também discutiremos o princípio da superposição, a medição de estados e o colapso da função de onda, além das portas lógicas quânticas fundamentais que operam em qubits únicos.

Esses elementos serão implementados e testados na prática com o uso do Qiskit, marcando o início do nosso desenvolvimento de circuitos quânticos e seus algoritmos.

FIAP

REFERÊNCIAS

AXLER, S. **Linear Algebra Done Right**. 4. ed. 2025. Disponível em: <https://linear.axler.net/>. Acesso em: 05 mai. 2025

BOMBELLI, R. **L'Algebra Opera**. Bolonha: Giouanni Rossi, 1579. Disponível em: <https://archive.org/details/lalgebra00bomb/>. Acesso em: 03 mai. 2025

DESCARTES, R. **La Géométrie**. Paris: A. Hermann, 1886. Disponível em: <https://archive.org/details/lagomtrie00descuoft/>. Acesso em: 04 mai. 2025

LOPES, A.; VEDRAL, V. **Computação Quântica: Da teoria à implementação**. São Paulo: Blucher, 2019.

NAHIN, P. J. **An Imaginary Tale: The Story of $\sqrt{-1}$**. Nova Jersey: Princeton University Press, 1998.

NIELSEN, M. A.; CHUANG, I. L. **Quantum Computation and Quantum Information**. 10th anniversary ed. Cambridge: Cambridge University Press, 2010.

NUMPY Documentation. **NumPy**. 2024. Disponível em: <https://numpy.org/doc/2.2/>. Acesso em: 06 mai. 2025

Qiskit SDK API Documentation. **IBM Quantum Platform**. 2025. Disponível em: <https://quantum.cloud.ibm.com/docs/en/api/qiskit>. Acesso em: 24 jun. 2025

STEINBRUCH, A.; WINTERLE P. **Álgebra Linear**. São Paulo: McGraw-Hill, 1987.